Comparison of Clustering Methods

Lee Wei Qing

Master of Science in Data Science and Analytics

The University of Leeds, School of Mathematics

August 2025

Abstract

Clustering is a major technique in machine learning that discovers patterns in data and groups them according to similarities without prior knowledge of class assignments. The performance of clustering algorithms can be influenced by data characteristics such as noise, dimensionality, sparsity, and separability. This study aims to evaluate four different approaches of clustering algorithms on three datasets: K-Means, DBSCAN, Agglomerative Clustering, and Gaussian Mixture Models (GMM). Synthetic data provides controlled experiments with known ground truth, the Spotify track features represent a real-world dataset of high dimensionality, and the MNIST handwritten digits images offer a challenging case of sparse and overlapping structure. The quality of clustering algorithms is measured by internal validation metrics, including Silhouette score, Davies Bouldin Index, and Calinski Harabasz Index, as well as external validation metrics such as Adjusted Rand Index and Normalised Mutual Information. Principal Component Analysis (PCA) was applied for dimensionality reduction.

According to the experiments carried out, K-Means, Agglomerative clustering, and GMM are relatively stable and produce interpretable clusters, whereas DBSCAN struggles when the dimensionality is increased and is highly sensitive to parameter tuning. Dimensionality reduction is a pivotal aspect, resulting in a significant improvement in reducing the Spotify dataset to two dimensions. In contrast, on the MNIST dataset, GMM achieves the best alignment with true digit labels at nine components. However, compressing the data into very low dimensions improved the silhouette scores at the expense of consistency with ground truth. Collectively, the results indicate that no universal best algorithm exists, and the suitability of an algorithm is determined based on the properties of the dataset, providing practical implications in both controlled and real-world settings.

Contents

1 Introduction **1**
1.1 Background And Motivation . . . 1
1.2 Aim And Objectives . . . 2
1.3 Document Overview . . . 3

2 Clustering **4**
2.1 K-Means . . . 5
2.2 Density-Based Spatial Clustering of Applications with Noise (DBSCAN) . . . 6
2.3 Hierarchical Clustering . . . 7
2.4 Gaussian Mixture Model . . . 8
2.5 Dimensionality Reduction . . . 10
2.6 Choosing the Optimal Number of Clusters . . . 10
2.6.1 Elbow Method . . . 11
2.6.2 Silhouette Analysis . . . 11
2.6.3 Gap Statistics . . . 12
2.6.4 Dendrogram . . . 13
2.7 Clustering Validation Metrics . . . 14
2.7.1 Internal Evaluation Metrics . . . 14
2.7.2 External Evaluation Metrics . . . 17

3 Synthetic Data **20**
3.1 Experiment 1: Optimal k Detection with Gaussian Blobs . . . 20
3.2 Experiment 2: Effect of Noise on Clustering . . . 22

4 Spotify Data **25**
4.1 Data Description . . . 25
4.2 Data Preprocessing . . . 25
4.3 Clustering Algorithms Applied . . . 26
4.3.1 Determining the Number of Clusters in K-Means and GMM . . . 27
4.3.2 Parameter Tuning in DBSCAN . . . 28
4.3.3 Linkage Selection in Agglomerative Clustering . . . 29
4.3.4 Covariance Structures in Gaussian Mixture Models . . . 30
4.4 Comparative Analysis . . . 31

5 MNIST Data **34**
5.1 Data Description . . . 34
5.2 Data Preprocessing . . . 35
5.3 Clustering Algorithms Applied . . . 35

5.3.1 Determining the Number of Clusters in K-Means and GMM 36
5.3.2 Parameter Tuning in DBSCAN 36
5.3.3 Linkage Selection in Agglomerative Clustering 37
5.3.4 Covariance Structures in Gaussian Mixture Models 38
5.4 Comparative Analysis 39

6 Discussion **42**
6.1 Influence of Dataset Characteristics 42
6.2 Role of Dimensionality Reduction 43
6.3 Practical Implications 43
6.4 Overview of Algorithm Capabilities and Limitations 44

7 Conclusions & Future Work **45**
7.1 Conclusion 45
7.2 Limitations & Future Work 46

List of Figures

2.1 Simulated dataset with three well-separated groups (left), and the elbow method plot (right) indicating the optimal number of clusters at $k = 3$. 11
2.2 Silhouette analysis indicates that the highest average silhouette score is obtained with $k = 3$. 12
2.3 Gap statistic plot of the expected log within-cluster dispersion against observed data, where the largest gap with 1-standard error rule resulted at $k = 3$. 13

3.1 Elbow plot for synthetic dataset with $k = 4$ clusters. 21
3.2 Synthetic dataset with $k = 10$ clusters (left), and the corresponding Gap Statistic plot (right), which identifies $k = 10$ as the optimal number of clusters. 22
3.3 Clustering results of K-Means, DBSCAN (fixed and best parameters), Agglomerative (Ward), and GMM with the addition of noise dimensions (top row) and within-dimension noise (bottom row). The performance is measured by Silhouette Score (left column) and Adjusted Rand Index (right column). The results are given as mean $\pm$ SD in 20 runs. 24

4.1 Distributions of numerical audio features in the Spotify dataset. 26
4.2 Correlation matrix of numerical audio features in the Spotify dataset. 27
4.3 Explained variance of principal components for the Spotify dataset, showing 95% of the data variance is around 7 components. 27
4.4 Elbow method for K-Means on the Spotify dataset. 27
4.5 k-distance plot for the Spotify dataset with *minPts* = 20. 28
4.6 Agglomerative clustering dendrograms using complete (top left), average (top right), Ward's (bottom left), and single (bottom right)linkage methods. 30
4.7 Heatmaps of Silhouette scores, DBI (top), and CHI (middle) for GMM (bottom) under different covariance structures and cluster components. 31
4.8 Clustering results on 2-dimensional Spotify data on K-Means (left) and GMM (right). 32
4.9 DBSCAN clustering result on the full-dimensional Spotify data, where the grey points represent noise. 33
4.10 Cluster feature means for K-Means on the Spotify dataset, normalised to [0.1,1]. 33

5.1 Examples of handwritten digit images with digit labels from the MNIST dataset. 34
5.2 Cumulative explained variance for PCA components on the MNIST dataset, showing that 95% of the data variance is around 150 components. 35
5.3 Elbow method (left) and Silhouette scores (right) for K-Means across different dimensions. 36
5.4 Heatmaps of ARI (top), NMI (middle), and Silhouette scores (bottom) for GMM under different covariance structures and data dimensions. 39

5.5 GMM clustering on 9-dimensional MNIST projected to 2D for visualisation. . 40
5.6 Confusion matrix of true digit labels versus GMM cluster labels on 9-dimensional MNIST. 41
5.7 Examples of digit images from Cluster 8, showing predominantly digit of “1”. . 41
5.8 Examples of digit images from Cluster 6, showing mixed digits of “5,” “3,” and “8”. 41

List of Tables

4.1 Cluster validity indices for K-Means on the Spotify dataset 28
4.2 DBSCAN results on full-dimensional Spotify data 29
4.3 DBSCAN results on 2D PCA-reduced Spotify data 29
4.4 DBSCAN results on 7D PCA-reduced Spotify data 29
4.5 Comparative clustering performance on Spotify across selected PCA dimensions. 32

5.1 DBSCAN results on full 784-dimensional MNIST data 37
5.2 DBSCAN results on 2D PCA-reduced MNIST data 37
5.3 DBSCAN results on 150D PCA-reduced MNIST data 37
5.4 Agglomerative clustering results with different linkage criteria on MNIST data 38
5.5 Comparative clustering performance on MNIST across selected PCA dimensions. 40

6.1 Comparative Algorithm Performance 44

Chapter 1

Introduction

1.1 Background And Motivation

Over the last few decades, variants of clustering algorithms have been suggested with a large spectrum of theoretical foundations and defined to solve, or at least achieve, particular data and cluster descriptions (Jain, 2010). Clustering is a branch of unsupervised learning in the aspect of machine learning that is used for discovering structures or inherent groups within unlabelled data. Fundamentally, clustering works by classifying unlabelled data into separate groups depending on similarity, usually without external training. Nowadays, clustering is utilized in various fields of applications, such as market segmentation (Wedel & Kamakura, 2000), music information retrieval (Liu et al., 2008), image segmentation (Coleman & Andrews, 1979), and bioinformatics (Yeung & Ruzzo, 2001).

Regardless of the effectiveness, there is no single clustering algorithm that can fit all the ideal solutions (Xu & Wunsch, 2005). The behaviour of clustering algorithms can be affected by the data structure characteristics, and the optimal number of clusters has become an issue (Milligan & Cooper, 1985). For example, the effectiveness of distance-based methods is diminished in high-dimensional space due to the so-called curse of dimensionality (Aggarwal et al., 2001). There is also a problem with noisy and overlapping structural features, which negatively impact the clustering performance (Omran et al., 2007). Images are often formed by a large number of pixels, which presents as high-dimensional data and requires dimensionality reduction before applying the clustering (van der Maaten & Hinton, 2008). As a result, choosing the right clustering algorithm is important and requires careful consideration. Different clustering algorithms carry distinct sets of assumptions and strengths. K-Means is efficient and scalable, but with the assumption that clusters are in the shape of a sphere of similar size (MacQueen, 1967). DBSCAN is a density-based algorithm that is used to find arbitrary-shaped clusters and handle the outliers, but is prone to parameter choice (Ester et al., 1996). Agglomerative clustering offers a tree-like structure, a dendrogram for interpretation, at the cost of computing efficiency (Rokach & Maimon, 2005). GMM is a generalization of the K-Means, which introduces probabilistic assignments and modelling of elliptical clusters with different covariance structures (Xu & Jordan,

1996).

Under these trade-offs, comparative studies are essential to carry out. The objective of this paper is to explore some of the most widely used clustering algorithms, which turned out to be K-Means, Density-Based Spatial Clustering of Applications with Noise (DBSCAN), Agglomerative clustering, and Gaussian Mixture Models (GMM). These approaches have been, to a large degree, studied, optimized, and compared both within scholarly literature and industrial deployments because they are effective for a wide range of data distributions, scale to large data, and vary in the usefulness to manage noise and irregular shapes of clusters (Xu & Wunsch, 2008; Rokach & Maimon, 2005). To this effect, three datasets have been selected to represent various problems. Synthetic data is used in this paper, which serves as a controlled setting to examine the robustness of clustering algorithms to noise and irrelevant features. The Spotify dataset represents a real-life use case that is adopted due to high-dimensional track features, which enable track recommendations. The MNIST handwritten digits dataset is utilized based on the large-scale and high-dimensional, conjoining the complexities of sparsity, overlap, and scale.

In this way, the study aims to emphasize both the strengths and weaknesses of each of the algorithms, with also the relevance of dimensionality reduction and the impact of the dataset characteristics.

1.2 Aim And Objectives

The general goal of this paper is to assess and compare the performance of those clustering algorithms on datasets of different complexity, and evaluate them using the validation metrics, while investigating the influence of dataset characteristics and dimensionality reduction. The specified objectives are:

1. To ascertain the hypotheses of whether techniques of k-selection in K-Means and GMM can correctly identify the number of clusters using synthetic data.

2. To evaluate the stability of K-Means, DBSCAN, agglomerative clustering, and GMM on synthetically generated data where the separability, noise, and irrelevant features are controlled.

3. To examine clustering performance on a high-dimensional real dataset using external validation metrics, the analysis is tested on the Spotify data set.

4. To assess the clustering accuracy on the MNIST data set, which is a large-scale and high-dimensional dataset, using both the internal and external validation metrics.

5. To conduct a comparative discussion of the abilities and limitations of clustering algorithms, identifying their practical implications in various application settings.

1.3 Document Overview

This study consists of seven chapters. Chapter 1 is an introductory chapter for the project, which describes the motivation, background, and objectives of the study while also giving a brief outline of its contents. Chapter 2 provides an overview of the theoretical background behind the clustering algorithms, optimal number of clusters techniques, and validation metrics. Chapter 3 conducts experiments with synthetic data, including inspecting k-selection methods and the behaviour of the algorithms in different levels of noise and irrelevant dimensions. Chapter 4 applies the clustering algorithms to analyse the Spotify dataset, examining how dimensionality reduction affects the quality of clustering results. Chapter 5 focuses on the MNIST handwritten digits dataset to test the performance of algorithms in sparse and overlapping data. Chapter 6 summarizes and contrasts the results in the three datasets and indicates the potentials and shortcomings of the algorithms. Lastly, Chapter 7 concludes the paper by summarising the main findings, discussing of limitations, and providing future possible work.

The implementation reported in Chapter 3, as well as in Chapters 4 and 5, is performed using Python within the Jupyter Notebook environment. The configuration is built on Python 3.11.3 with the incorporation of common libraries including `NumPy`, `pandas`, `scikit-learn`, `Matplotlib`, and `Seaborn`. All the source code is available in a public GitHub repository [1]so that the results are fully reproducible. The development is carried out on the system that runs Windows 11, Version 23H2, using an AMD Ryzen 7 6800H and 24.0 GB of RAM installed. All experiments in the paper utilized the same environment and the same version of the software to allow comparability.

[1]https://github.com/lwq1023999/Clustering-Technique-A-Comprehensive-Overview

Chapter 2

Clustering

Cluster analysis is a technique in unsupervised machine learning, where data points are grouped based on their inherent similarities without relying on prior-established classes or labels. Since clustering inception, it has been a method for discovering latent structure, patterns, or meaningful subgroups in unlabelled datasets. It achieves convergence by maximizing the similarities among data points within the same cluster (intra-cluster similarity) and dissimilarities between data points from different clusters (inter-cluster dissimilarity) (Rokach & Maimon, 2005; Xu & Wunsch, 2008). In contrast to supervised-learning methods such as classification and regression, which require a class-labelled dataset to train models, clustering operates entirely in an unsupervised setting, which only requires the dataset without knowledge of group labels (Xu & Wunsch, 2008). To quantify the relationship between data points, clustering algorithms employ various similarity or distance metrics, such as Euclidean, Manhattan distance, or cosine similarity, based on different approaches such as partitioning, density, hierarchy, or probabilistic modelling (Rokach & Maimon, 2005; Omran et al., 2007; Xu & Wunsch, 2008; Xu & Tian, 2015).

Over the past few years, various types of clustering algorithms have evolved and been proposed, often using different assumptions and approaches to achieve their goals (Xu & Wunsch, 2008). This paper focuses on four different clustering algorithms: K-Means (partition-based), Density-Based Spatial Clustering of Applications with Noise (DBSCAN; density-based), Hierarchical Clustering, and Gaussian Mixture Models (GMM; probabilistic model-based). Partition-based methods divide data into a pre-determined number of non-overlapping categories, with each datum falling under the jurisdiction of one category based on similarity . Density-based methods group data points according to density, where continuous regions of high-density form clusters, while sparse regions are treated as noise. Hierarchical-based methods build clusters in a tree structure by ranking the clusters across different levels of hierarchy. Model-based methods group data points, assuming the data is generated from a mixture of probability distributions.

In this chapter, the theoretical background of each algorithm will be discussed, followed by dimensionality reduction techniques, methods for identifying the optimal number of clusters, and validation metrics, providing a comprehensive understanding of their applicability.

2.1 K-Means

In the modern reality of data analysis, K-Means clustering is one of the most utilized partitioning algorithms in unsupervised learning. First coined by MacQueen in 1967, the algorithm has become a foundational tool in data mining and pattern recognition, due to its conceptual simplicity, lightweight computations, and good performance in handling massive data sets (MacQueen, 1967; Xu & Wunsch, 2008; Jain, 2010). The primary aim of K-Means is to partition data into K mutually exclusive and exhaustive clusters. Before running the algorithm, the number of clusters K need to be specified by the user. After the determination of K, the algorithm positions the cluster centroids (means) iteratively to minimize the global clustering error, where data points in each cluster are centred about the centroid. The global clustering error is measured by the within-cluster sum of squares (WCSS), or is generally considered as the Sum of Squared Errors (SSE) or simply inertia (Rokach & Maimon, 2005; Omran et al., 2007; Jain, 2010). Given K clusters $C_1, C_2, \ldots, C_K$ with centroids $\mu_1, \mu_2, \ldots, \mu_K$, the objective function is expressed as:

$$J = \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2, \tag{2.1}$$

where C_k denotes the k-th cluster, μ_k is the centroid of that cluster, and x is the data point belonging to C_k. In simple words, J adds up squared distances from each data point x to the centre of its assigned cluster, where lower J value indicates that the clusters are more compact and cohesive. Essentially, the K-Means algorithm iterates through the following steps (Rokach & Maimon, 2005; Xu & Wunsch, 2008; Jain, 2010):

1. Randomly select the predefined K points as the initial cluster centroids.

2. Assign each data point to the nearest centroid by comparing distances to all K centroids, normally measured using Euclidean distance

$$d(x, \mu_k) = \sqrt{\sum_{i=1}^{m} (x_i - \mu_{k,i})^2} \tag{2.2}$$

   where $x = (x_1, x_2, \ldots, x_m)$ is a data point with m features, and $\mu_k = (\mu_{k,1}, \mu_{k,2}, \ldots, \mu_{k,m})$ is the centroid of cluster C_k, to form K clusters (Jain, 2010).

3. Once all the points are assigned to their closest centroids, the centroids are recalculated by computing the mean values of all points found within each cluster. This is to relocate the centroids to new centre positions to improve the cluster representation, reducing the clustering error (2.1).

4. Repeat step 2 and step 3 until the cluster assignments stabilize or the centroids do not change significantly between consecutive iterations, producing the final set of clusters.

Although convergence is ensured, the result may be an inferior local minimum due to the greediness of the algorithm itself and the initial random placement of the centroids. The time complexity for K-Means is given by $O(T \cdot K \cdot N \cdot d)$. Thus, computational time grows linearly and is affected by iterations T, clusters K, data points N, and number of dimensions d (Rokach & Maimon, 2005; Xu & Wunsch, 2008). Such linearity guarantees the effectiveness of the algorithm and is suitable for large-scale data clustering. Besides being simple to implement, the results also exhibit strong intuitive elements, which are easy to understand and interpretable.

Despite its tremendous applications, K-means exhibits several notable limitations. A primary drawback is the sensitivity to the initialization of centroids. Poor initialization will lead to convergence at suboptimal local minima, especially in high-dimensional or sparse datasets. To mitigate this, the K-Means++ algorithm was proposed, by employing the probabilistic seeding method in which initial centroids are selected to be far apart from each other to increase clustering accuracy and stability (Arthur & Vassilvitskii, 2006). Another limitation being the assumption that data are spherical and of roughly equal size. This may result in poor results on datasets consisting of elongated, irregular, or density-varying clusters. Furthermore, K-Means is sensitive to outliers and noise, as the squared distance objective function can disproportionately weight distant points (Xu & Wunsch, 2008). Moreover, the performance of K-Means degrades in the presence of overlapping clusters or when the number of clusters is not well-defined, and its rigid assumptions limit effectiveness on complex, non-globular datasets despite its linear scalability. Nevertheless, K-Means remains a popular baseline algorithm in clustering research and is frequently used as a benchmark to evaluate more advanced and flexible clustering techniques.

2.2 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

DBSCAN is a density-based clustering algorithm that does not require prior knowledge of the number of clusters, unlike K-Means. It groups high-density points into a cluster, separating them from low-density areas, which is suitable for identifying clusters of arbitrary shape (Ester et al., 1996; Rokach & Maimon, 2005; Xu & Tian, 2015). Two key parameters are required for DBSCAN: ε (epsilon) and `MinPts`. ε is the maximum distance where data points fall within this distance are considered in the same neighbourhood. `MinPts` is the minimum number of data points to form the neighbourhood. Based on these parameters, data points are classified as core points when there are at least `MinPts` points that fall within their ε-radius neighbourhood. Border points are within the neighbourhood of a core point but do not meet the `MinPts` threshold. Noise points are neither core nor border points, often treated as outliers. DBSCAN works as follows (Ester et al., 1996; Xu & Wunsch, 2008):

1. Pick an arbitrary unvisited point as the initial point and find all points within the ε distance to form its neighbourhood.

2. If the neighbourhood contains at least `MinPts` points, a new cluster is formed. The initial point is marked as a core point. If it has fewer than `MinPts`, the point is temporarily marked as noise (this may change later).

3. Expand the cluster by searching the neighbourhood of each core point and adding its neighbours to the cluster. Points in the neighbourhood that are not core points will be marked as border points.

4. Repeat Steps 2 and 3 until all points have been visited and all clusters are identified.

5. Points that remain labelled as noise after the process are considered noise or outliers.

In simple terms, DBSCAN clusters all core points with non-core points whose density can directly reach the core points and classifies any point whose density cannot reach the core points in low-density areas as noise. This process enables DBSCAN to find clusters with irregular and complex shapes while filtering out noise.

DBSCAN is particularly effective in applications involving spatial data, anomaly detection, and natural group identification in unstructured datasets. However, it is sensitive to selecting optimal parameters when dealing with different densities in a dataset. An overly large ε can merge clusters into one, while a too small ε can fragment clusters into multiple smaller ones. In addition, when dealing with high-dimensional data, its effectiveness can be diminished due to the curse of dimensionality (Jain, 2010; Xu & Tian, 2015). For large dataset, its performance may degrade since its time complexity is $O(n^2)$ in the worst case, and its memory usage is high (Bindra & Mishra, 2017). To address these limitations, a hierarchical density-based clustering method, HDBSCAN, is developed, offering greater robustness to parameter selection and varying densities within datasets.

2.3 Hierarchical Clustering

Hierarchical clustering organizes data into a tree-based nested cluster structure. Hierarchical clustering has two algorithms to form clusters: agglomerative and divisive. Agglomerative is a bottom-up approach, where each data point begins as a single cluster and merges the individual points iteratively, and combines them into larger clusters. Divisive is a top-down approach, where all data points start in one cluster and are split into smaller clusters until each data point is an individual cluster (Jain et al., 1999; Rokach & Maimon, 2005; Xu & Wunsch, 2008). In this study, the agglomerative clustering will be used due to its simplicity and computational efficiency compared to divisive clustering. The workflow of agglomerative clustering is as follows (Xu & Wunsch, 2008):

1. Serves each data point as a separate cluster.

2. Computes the pairwise distance between each of the clusters using a distance matrix, also known as linkage criteria, based on the distance function in (2.2). There are mainly four linkage criteria to choose from (Rokach & Maimon, 2005; Xu & Wunsch, 2008).

   - Single Linkage: The distance between the closest pair of points from different clusters.

$$d_{\text{single}}(C_k, C_\ell) = \min\{d(x, y) \mid x \in C_k,\ y \in C_\ell\} \tag{2.3}$$

   - Complete Linkage: The distance between the farthest pair of points from different clusters.

$$d_{\text{complete}}(C_k, C_\ell) = \max\{d(x, y) \mid x \in C_k,\ y \in C_\ell\} \tag{2.4}$$

   - Average Linkage: The average distance between all points from different clusters.

$$d_{\text{average}}(C_k, C_\ell) = \frac{1}{|C_k|\,|C_\ell|} \sum_{x \in C_k} \sum_{y \in C_\ell} d(x, y) \tag{2.5}$$

   - Ward's Linkage: The distance that minimizes the total within-cluster sum of squares defined in (2.1).

$$d_{\text{ward}}(C_k, C_\ell) = \frac{|C_k|\,|C_\ell|}{|C_k| + |C_\ell|} \cdot \|\mu_{C_k} - \mu_{C_\ell}\|^2 \tag{2.6}$$

3. Merges clusters iteratively based on the chosen linkage criterion.

4. Update the distance matrix by recalculating the distance between the new cluster and the other clusters.

5. Repeat steps 3 and 4 until all clusters are merged into a single cluster.

The result of the clustering is often visualized in a tree-like diagram, known as a dendrogram. The leaf nodes corresponded to each single data point, and the internal nodes corresponded to the clusters. To find the ideal number of clusters, the dendrogram can be cut at the desired height. This means hierarchical clustering does not require a predefined number of clusters (Murtagh & Contreras, 2012). However, hierarchical clustering is computationally expensive, with the time complexity of $O(n^2 \log n)$, which is not suitable for large datasets. Another limitation is that the results are sensitive to the choice of linkage criteria, where different linkage criteria produce different cluster structures. It is also sensitive to noise and outliers, and once clusters are merged, there is no backtracking (Rokach & Maimon, 2005).

2.4 Gaussian Mixture Model

Gaussian Mixture Model (GMM) is a probabilistic model-based clustering algorithm, where data is represented as a mixture model that is a weighted combination of a fixed number of

Gaussian distributions. Each cluster is represented by Gaussian distribution parameters: the mean μ_k (centre), the covariance Σ_k (spread), and the mixing coefficient π_k (weight) (Xu & Jordan, 1996; McLachlan et al., 2019). GMM computes the probability of each data point belonging to all clusters, and the cluster with the highest probability is assigned as the point label. This approach is known as soft clustering, which benefits a more general modelling of complex data where cluster boundaries may be uncertain or have an overlapping nature (Bishop, 2006). Let x_n denote the n-th data point in the dataset (previously denoted as x). The overall probability of a data point x_n under the mixture model is:

$$P(x_n) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k) \tag{2.7}$$

where π_k is the mixing coefficient of cluster k, and $\mathcal{N}(x_n \mid \mu_k, \Sigma_k)$ is the Gaussian probability density function for cluster k with mean μ_k and covariance Σ_k (Bishop, 2006; Deng & Han, 2018). This expression is called marginal likelihood, which sums the contributions from all Gaussian components to determine the probability of observing the sample x_n under the whole mixture. The probability that x_n belongs to cluster k is given by:

$$P(z_n = k \mid x_n) = \frac{\pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \cdot \mathcal{N}(x_n \mid \mu_j, \Sigma_j)} \tag{2.8}$$

where the numerator computes the likelihood of x_n under Gaussian k weighted by its mixing coefficient, whereas the denominator corresponds to the marginal probability $P(x_n)$ from the (2.7) (Bishop, 2006; Deng & Han, 2018). This expression is known as the posterior probability.

The Expectation-Maximization (EM) algorithm is used to estimate the GMM parameters, coefficients, means, and covariances. EM operates iteratively through the following steps (Xu & Jordan, 1996; Bishop, 2006; McLachlan et al., 2019):

- Expectation step (E-step): Compute the posterior probabilities for each data point belonging to each Gaussian component, given current parameter estimates.
- Maximization step (M-step): Update the parameters to maximize the expected log-likelihood of the data under the current assignment probabilities.

These two steps are repeated until convergence occurs, where the parameters no longer exhibit any significant changes from one iteration to the next.

GMM is more flexible compared to traditional clustering, partitioning, and hierarchical clustering. Such a model can fit clusters that are elliptical with variation in size and orientation. This is suitable for data where the underlying distribution is close to Gaussian but not necessarily spherical. Despite its strength, GMM has some limitations. The algorithm is sensitive to parameter initialization, which can lead to convergence at suboptimal local optima (Xu & Jordan, 1996). Furthermore, a large number of components causes increased computational complexity and raises the risk of overfitting, especially in high-dimensional spaces or small datasets (Deng

& Han, 2018). To address this, Variational Bayesian GMMs and other advanced variants have been proposed. These approaches perform Bayesian inference to determine distributions over the model parameters, thus providing greater robustness to initialization and avoiding overfitting by adaptive model complexity (Bishop, 2006).

2.5 Dimensionality Reduction

In practice, data is usually in high dimensions and contains a multitude of features, many of which may be redundant or even contribute to certain noise. This exposes problems as clustering becomes challenging in high-dimensional space due to the curse of dimensionality (Bishop, 2006; Xu & Wunsch, 2008). This essentially makes the distances between points less meaningful and increases the complexity expenses exponentially. To solve this problem, dimensionality reduction techniques try to construct a low-dimensional space where the data can be mapped while preserving the most informative structure within it. This enables clustering algorithms to perform better with a higher separation of clusters. Another use of this dimensionality reduction is to visualize the high-dimensional data in two- or three-dimensional space.

Some common methods for dimensionality reduction are Principal Component Analysis (PCA), t-distributed Stochastic Neighbour Embedding (t-SNE), and Uniform Manifold Approximation and Projection (UMAP). PCA is a linear method where data is transformed into orthogonal dimensions that maximize variance and is often used in clustering as a preprocessing step (Xu & Wunsch, 2008). Since it is linear, PCA is unable to deal with non-linear correlations in data. The t-SNE is a non-linear dimensionality reduction method that attempts to preserve local neighbourhood by mapping high-dimensional data into two or three dimensions and is often used for visualisation (Wattenberg et al., 2016). However, t-SNE is time-consuming and hyperparameter-sensitive, which can produce inconsistent results across repeated operations. The more recent method is UMAP, which is based on manifold learning that tries to preserve local and global structure more effectively, resulting in faster computation and greater scalability when compared to t-SNE (McInnes et al., 2018). However, dimensionality reduction is always associated with information loss, which may influence the quality of downstream clustering (Xu & Wunsch, 2008). This paper deploys PCA due to its ease of use, interpretability, and its power as a preprocessing method for clustering.

2.6 Choosing the Optimal Number of Clusters

The selection of the number of clusters is an important task in cluster analysis. In partition-based methods, such as K-Means, and model-based methods, such as GMM, often required a predefined number of clusters k. A poor choice may lead to underfitting or overfitting the clustering results. There is no best solution for choosing the number of clusters. The optimal k depends on the data structure, clustering algorithm to be used, and the objective of the analysis (Mirkin,

2011). This study will discuss several techniques, including the elbow method, silhouette analysis, gap statistics, and dendrogram for hierarchical clustering. Each method offers different perspectives under visual heuristics and statistical approaches.

2.6.1 Elbow Method

Elbow method is a common and intuitive technique for estimating the number of clusters, k, attributed to Thorndike. This method is based on running K-means algorithms for a range of candidate values k and plotting SSE defined in (2.1) against the number of clusters, k (Thorndike, 1953; Humaira & Rasyidah, 2018). The SSE naturally decreases as k increases, as more clusters reduce the distance between the points and their assigned cluster. Beyond a certain point, there will be only a small or no improvement. This point resembles the 'elbow' in the graph, which is considered optimal k (Syakur et al., 2018; Bholowalia & Kumar, 2014). It strikes a balance between cluster compactness and model simplicity. However, identifying the optimal k can be ambiguous in practice when the elbow point is hardly visible in the curve (Syakur et al., 2018). Despite this, the elbow method remains a strong first-choice heuristic for initial cluster number estimation due to its conceptual simplicity. Figure 2.1 shows that the simulated data are clearly clustered into three distinct groups (left) and that there is a sharp bend at $k = 3$ (right), which indicates the optimal number of clusters.

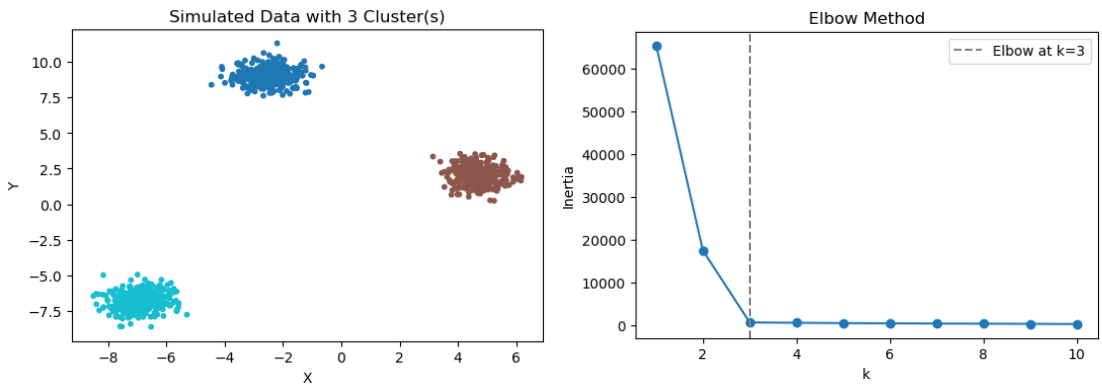


Figure 2.1: Simulated dataset with three well-separated groups (left), and the elbow method plot (right) indicating the optimal number of clusters at $k = 3$.

2.6.2 Silhouette Analysis

The Silhouette analysis is another graphical and analytical technique to determine the optimal number of clusters k in unsupervised learning. First introduced by Rousseeuw, assesses how each data point fits within its assigned clusters compared to other clusters. For a given number of clusters k, the Silhouette coefficient defined in (2.11) is computed, ranging from -1 to 1. Plotting the Silhouette score over different values of k, where a high Silhouette score indicates that the clusters are well separated and internally cohesive, to help determine the optimal number

of clusters (Rousseeuw, 1987). Figure 2.2 presents an example of the silhouette scores results using the simulated data mentioned above, and the peak value appears at $k = 3$, reaffirming the identification of the most appropriate value of k. This method helps evaluate clustering structure without supervision and is less sensitive to the scale of features or initial centroids compared to methods like the elbow method. It has also been used effectively in ensemble clustering approaches, where multiple clustering solutions are combined and weighted according to their average Silhouette scores to yield more robust results. More recently, simplified variants have been proposed to reduce computational overhead, making the method practical for large-scale datasets (Wang et al., 2017). Additionally, researchers have found that this method performs reliably across a wide variety of clustering techniques and domains, offering consistent guidance on selecting the number of clusters (Rousseeuw, 1987).

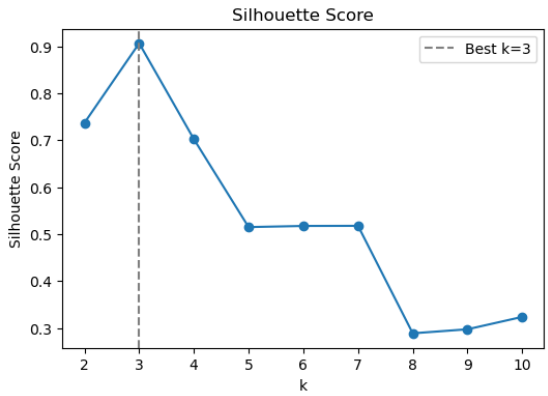


Figure 2.2: Silhouette analysis indicates that the highest average silhouette score is obtained with $k = 3$.

2.6.3 Gap Statistics

Gap Statistics is another method that provides a statistically principled approach for estimating the optimal number of clusters k, introduced by Tibshirani. This method was developed to address the limitations of traditional visual methods, such as the elbow method, which often require visual interpretation and may not reveal a clear structure. It compares the within-cluster dispersion of the observed data with the expected dispersion under the null reference distribution (Tibshirani et al., 2001; Hastie et al., 2009). The gap value is defined as

$$\text{Gap}(k) = \mathbb{E}[\log(W_k^*)] - \log(W_k), \tag{2.9}$$

where W_k is observed within-cluster dispersion, equivalent to the WCSS objective function defined in (2.1), and W_k^* is the corresponding value for reference datasets generated under a null model. Instead of choosing the k that maximizes the gap value, a selection rule based on

the "1-standard-error" criterion is proposed, choosing the smallest k such that:

$$\text{Gap}_n(k) \geq \text{Gap}_n(k+1) - s_{k+1}, \tag{2.10}$$

where s_{k+1} is the standard deviation of $\log(W_k^*)$ from the reference datasets, typically scales as $s_{k+1} = \sqrt{1 + \frac{1}{B}} \cdot \text{SD}_{k+1}$, where B is the number of bootstraps used (Tibshirani et al., 2001; Hastie et al., 2009).

This rule helps prevent overestimation of the number of clusters by favouring simpler models unless additional clusters offer a statistically significant gain. Although Gap Statistics provides a structured statistical procedure for estimating the number of clusters, it has certain limitations. It only performs well when the clusters are well-separated, and its effectiveness decreases when the data have overlapping clusters or lie near lower-dimensional subspaces (Tibshirani et al., 2001). In such cases, the notion of a "true" cluster becomes blurred, leading to unclear or inconsistent results. The Gap curve may exhibit multiple local maxima, complicating the selection of an appropriate value of k and reintroducing subjectivity. In addition, when data lie close to a lower-dimensional subspace, the use of a uniform reference distribution can distort the expected dispersion, reducing the accuracy (Tibshirani et al., 2001). This method is also computationally intensive due to repeated simulations of reference datasets. Figure 2.3 shows the results of the Gap Statistic applied to the same simulated data, with both the 1-standard error rule, and the maximum gap value, pointing to $k = 3$ as the optimal number of clusters.

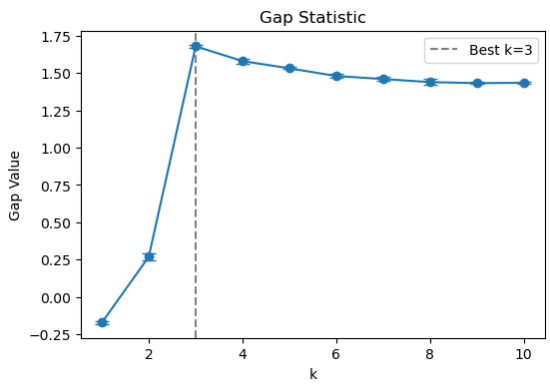


Figure 2.3: Gap statistic plot of the expected log within-cluster dispersion against observed data, where the largest gap with 1-standard error rule resulted at $k = 3$.

2.6.4 Dendrogram

The dendrogram is a visual tool used in hierarchical clustering to determine the optimal number of clusters. It is a tree-like structure constructed either by merging individual data points (agglomerative) or splitting a global cluster (divisive), illustrating the nested structure of data (Xu & Wunsch, 2008; Kaufman & Rousseeuw, 2009; Hastie et al., 2009). The dendrograms visually capture the evolution of cluster formation over different linkage methods, such as single,

complete, average, or Ward's method, with an example result shown in Section 4.3.3. Determining the number of clusters in a dendrogram is usually done by "cutting" the tree at a certain height, where the cutting point is the vertical distance between merges (linkage distances) that are longest and most distinct. This cut indicates the number of clusters that best represent the data without prior knowledge of k.

A key advantage of dendrogram-based methods is that they do not require the number of clusters to be specified in advance and provide intuitive and interpretable visual summaries of the clustering process (Kaufman & Rousseeuw, 2009). However, they are sensitive to noise and outliers and have high computational complexity, making them less efficient when dealing with large or high-dimensional datasets. While dendrograms work well for small and medium-sized hierarchical data, they can be difficult or even impossible to scale when applied to large-scale problems (Xu & Wunsch, 2008).

2.7 Clustering Validation Metrics

Clustering validation presents a distinctive challenge, yet a crucial step in unsupervised learning, especially when class labels (ground truth) are not available. As a result, numerous metrics have been proposed to assess the quality of clustering partitions, and mainly categorized into internal and external validation metrics (Xu & Wunsch, 2008; Liu et al., 2010; Arbelaitz et al., 2013). Internal metrics depend solely on the data itself and the cluster results, while external metrics require a known reference compared to the cluster results. These metrics help guide model selection and parameter tuning, providing insight into how well the algorithm aligns with given assumptions about the data structure (Arbelaitz et al., 2013). Most clustering validity indices are based on mathematical approaches derived from the concept of similarity, usually specified in terms of distances. Although the comparison of such indices may provide information on circumstances when one algorithm shows a particular advantage over the other, the outcomes thereof must not be treated as evidence of overall superiority. The choice of clustering evaluation metrics also depends on significant data and the availability of meaningful structure. In the absence of a real structure, the results of clustering can be mathematically sound, but cannot have any true interpretation. This problem is particularly severe when dealing with high-dimensional data, where distance-based measure tends to be unreliable. The following subsections outline internal and external validation methods, along with their respective strengths and limitations.

2.7.1 Internal Evaluation Metrics

Internal validation metrics evaluate the quality of clustering based solely on considering the dataset and the clustering output, without the need for external labels. These metrics typically measure intra-cluster compactness and inter-cluster separation through similar measures of distance (Xu & Wunsch, 2008; Liu et al., 2010; Arbelaitz et al., 2013). The commonly used metrics are Silhouette Score, Davies Bouldin Index, and Calinski-Harabasz Index. These in-

dices correspond to assumptions regarding desirable clustering properties, uniform density, or well-separated spherical shapes, but rarely hold in practice, with clusters commonly varying considerably in size, shape, and dispersal. A higher or lower score suggests that clustering agrees with assumed characteristics, but does not necessarily imply the presence of any significant structure. Also, the reliability of these metrics may decrease in high-dimensional spaces as the distances between data points become concentrated. This degrades the credibility of internal indices in this case. Hence, internal validation is recommended to be used within the same data set and feature space, preferably using several indices for more trustworthy and interpretable outcomes (Arbelaitz et al., 2013).

Silhouette Coefficient

The Silhouette Coefficient is a commonly used internal validation metric in cluster analysis (Rousseeuw, 1987; Kaufman & Rousseeuw, 2009). While silhouette analysis used the average scores of K clusters to determine the optimal number of clusters, the silhouette coefficient is used as a quantitative measure of how well each data point fits within its assigned cluster. For a given data point $x_i \in C_k$, the Silhouette Coefficient is defined as:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \tag{2.11}$$

where

$$a(i) = \frac{1}{|C_k| - 1} \sum_{\substack{x_j \in C_k \\ j \neq i}} d(x_i, x_j), \tag{2.12}$$

is the average distance from x_i to all other points in its own cluster C_k (cohesion), and

$$b(i) = \min_{\ell \neq k} \frac{1}{|C_\ell|} \sum_{x_j \in C_\ell} d(x_i, x_j), \tag{2.13}$$

is the smallest average distance from x_i to all points in any other cluster C_ℓ (separation). The silhouette score ranges from -1 to 1. Higher values of $s(i)$ closer to 1 indicate better-defined clusters, where the within-cluster distance $a(i)$ is much smaller than the nearest-cluster distance $b(i)$. On the other side, values of $s(i)$ close to -1 indicate poor clustering results. Values of $s(i)$ around 0 indicate the points located between clusters, suggesting overlapping clusters (Kaufman & Rousseeuw, 2009).

This metric is robust across the clustering methods, as it does not require any external labels, which is useful in unsupervised learning contexts where ground truth is unavailable. However, there are limitations to its computational complexity. This metric requires computing the pairwise distance between data points, which makes it computationally expensive for large-scale datasets (Liu et al., 2010; Arbelaitz et al., 2013). To address this, Simplified Silhouette methods have been proposed. These methods use centroid distance to approximate intra-cluster distance

and inter-cluster distance rather than full pointwise comparisons, which reduces overhead while maintaining competitive performance (Wang et al., 2017).

Davies–Bouldin Index

The Davies–Bouldin Index (DBI), introduced by Davies and Bouldin, is another internal clustering validation metric that evaluates the quality of clustering based on intra-cluster compactness and inter-cluster separation (Davies & Bouldin, 1979; Xu & Wunsch, 2008). This index quantifies the average similarity of each cluster to its most similar neighbour cluster by measuring the ratio of within-cluster dispersion to between-cluster distances. Let K be the total number of clusters. The formula of DBI is defined as

$$\text{DBI} = \frac{1}{K} \sum_{k=1}^{K} \max_{\ell \neq k} \left(\frac{S_k + S_\ell}{d(\mu_k, \mu_\ell)} \right), \tag{2.14}$$

where

$$S_k = \frac{1}{|C_k|} \sum_{x \in C_k} d(x, \mu_k), \tag{2.15}$$

is the average distance between each point in cluster C_k and its centroid μ_k, and $d(\mu_k, \mu_\ell)$ is the distance between the centroids of clusters C_k and C_ℓ, computed using the Euclidean metric defined in (2.2). A lower DBI value indicates better clustering performance, with compact and well-separated clusters.

DBI is computationally efficient, making it suitable for real-time applications, compared to more complex indices like Silhouette coefficient. It assumes that clusters are spherical and uses Euclidean distance to calculate both compactness and separation, performing well when clusters are globular and equal in size. However, when clusters have irregular shapes, uneven densities, or there is overlapping clusters, the accuracy declines significantly (Arbelaitz et al., 2013; Ros et al., 2023). Additionally, it is sensitive to outliers, which can distort the result and produce misleading evaluations. To overcome these limitations, the Partitioning Davies–Bouldin Index (PDBI) is proposed to enhance DBI by incorporating local structure, improving robustness in complex clustering scenarios (Ros et al., 2023).

Calinski-Harabasz Index

Calinski and Harabasz proposed the Calinski–Harabasz Index (CHI), which is one of the widely used internal clustering validation metrics, also known as the Variance Ratio Criterion (Calinski & Harabasz, 1974; Xu & Wunsch, 2008). It evaluates the clustering performance based on the ratio between-cluster dispersion and within-cluster dispersion. The index is defined as

$$CH(K) = \frac{B_K}{W_K} \cdot \frac{N - K}{K - 1}, \tag{2.16}$$

where K is the number of clusters, N is the total number of data points, B_K is the between-cluster scatter matrix, defined as

$$B_K = \sum_{k=1}^{K} n_k(\mu_k - \mu)(\mu_k - \mu)^T, \tag{2.17}$$

which captures the separation between clusters, where μ_k is the centroid of cluster C_k, n_k is the number of points in cluster C_k, and μ is the global mean of the data. W_K is the within-cluster scatter matrix, defined as

$$W_K = \sum_{k=1}^{K} \sum_{x \in C_k} (x - \mu_k)(x - \mu_k)^T, \tag{2.18}$$

which captures the compactness within each cluster, where x is a data point in cluster C_k, and μ_k is the centroid of the corresponding cluster. A higher CH index indicates that clusters are more cohesive internally and better separated, corresponding to large B_k and small W_k values.

This index has strengths for its simplicity and effectiveness, offering consistent performance while remaining computationally efficient compared to other validation metrics (Xu & Wunsch, 2008). However, it is sensitive to outliers and performs unstably with datasets that have noise or skewed distributions, making it less suitable for capturing non-convex or complex cluster structures (Arbelaitz et al., 2013).

2.7.2 External Evaluation Metrics

External validation metrics gauge clustering quality by comparing predicted cluster labels against the known ground-truth labels. Metrics such as the Adjusted Rand Index and Normalized Mutual Information measure the level of agreement between the clustering output and the true labels, directly indicating how well the algorithm recovers the underlying structure (Xu & Wunsch, 2008). However, these measures require labelled data, which is mostly unapplicable in the case of unsupervised learning in the real world.

Adjusted Rand Index

The Adjusted Rand Index (ARI), originated by Hubert and Arabie, is a metric for external validation in clustering analysis, particularly when the true class labels (ground truth) are available (Hubert & Arabie, 1985). ARI index measures the agreement between two data partitions while correcting for chance. It extends the Rand Index (RI), which evaluates how consistently two partitions assign all pairs of samples. ARI improves on RI by adjusting its expected value under a random model, ensuring that a score of zero reflects clustering similarity that is no better than chance. Let a, b, c, d represent the number of object pairs with the following relationships between a ground truth partition $C = \{C_1, \ldots, C_r\}$ and a predicted cluster $K = \{K_1, \ldots, K_s\}$.

- a: both objects with the same labels in C and in the same cluster in K,

- b: both objects with the same labels in C but in different clusters in K,
- c: objects with different labels in C but in the same cluster in K,
- d: objects with different labels in C and in different clusters in K.

The Rand Index is defined as:

$$RI = \frac{a+d}{a+b+c+d} \tag{2.19}$$

This measures the proportion of correct decisions made over all possible pairs of data points, and does not account for agreement that might occur purely by chance (Rand, 1971; Xu & Wunsch, 2008). To address this, ARI is introduced to correct the RI in (2.19) by subtracting the expected similarity of random clustering and normalizing the result, and is defined as:

$$ARI = \frac{RI - \mathbb{E}[RI]}{1 - \mathbb{E}[RI]}, \tag{2.20}$$

where $\mathbb{E}[RI]$ is the expected Rand Index under random labelling. A more common formulation of ARI is

$$ARI = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \Big/ \binom{n}{2}\right]}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \Big/ \binom{n}{2}\right]}, \tag{2.21}$$

where $\binom{\cdot}{2}$ denotes the binomial coefficient, n_{ij} is the number of elements that are in cluster i of one partition and cluster j of the other, $a_i = \sum_j n_{ij}$ is the number of elements in cluster i of the first partition, $b_j = \sum_i n_{ij}$ is the number of elements in cluster j of the second partition, and n is the total number of data points.

The ARI ranges from -1 (complete disagreement) to 1 (perfect agreement), with 0 being the random similarity one would expect from random labelling (Hubert & Arabie, 1985; Vinh et al., 2010). ARI is suitable when the ground truth labels exist due to robustness and invariance to label designations. It works well in situations where the predicted clusters need to be compared against the known categories. However, ARI comes with some pitfalls. It is sensitive to large cluster size imbalances, tending to weigh big clusters more and small clusters less. It presupposes the true number of clusters to be known and fixed, so its performance deteriorates fast if the number of predicted clusters strays too far from the ground truth (Vinh et al., 2010). Furthermore, although it compensates for random agreement, it assumes an underlying random clustering scheme that may not hold in structured or constrained datasets (Hubert & Arabie, 1985). Additionally, it measures based on pairwise agreement and does not consider higher-order cluster structures, which are inapplicable to scenarios where the ground truth is unknown (Meilă, 2007).

Normalized Mutual Information

Normalized Mutual Information (NMI) is an external validation metric used to compare the similarity between two partitions of data based on the shared information (Meilă, 2007). Given

two partitions of data, $C = \{C_1, \ldots, C_r\}$ for the ground truth labels, and $K = \{K_1, \ldots, K_s\}$ for the predicted labels. Mutual information (MI) is defined:

$$I(C;K) = \sum_{i=1}^{|C|} \sum_{j=1}^{|K|} \frac{n_{ij}}{N} \log \left(\frac{n_{ij} \cdot N}{n_{i\cdot} \cdot n_{\cdot j}} \right), \tag{2.22}$$

where n_{ij} is the number of samples in both true cluster C_i and predicted cluster K_j, and N denotes the total number of samples. The $\frac{n_{ij}}{N}$ is often defined as the joint probability, while $\frac{n_{i\cdot}}{N}$ and $\frac{n_{\cdot j}}{N}$ correspond to the marginal probabilities in some articles. MI in (2.22) is then normalized to form NMI and defined:

$$\text{NMI}(C,K) = \frac{2 \cdot I(C;K)}{H(C) + H(K)}, \tag{2.23}$$

where $H(C)$ and $H(K)$ are the entropies of the clusters C and K. The entropy is defined:

$$H(X) = -\sum_{i=1}^{|X|} \frac{n_{i\cdot}}{N} \log_2 \left(\frac{n_{i\cdot}}{N} \right), \tag{2.24}$$

with $X \in \{C, K\}$, and usually measured using logarithm base 2.

The NMI values are within the interval [0,1], value of 1 indicates a perfect match between the two partitions, and 0 implies no mutual information. NMI is label-invariant, which is insensitive to permutations of cluster labels (Vinh et al., 2010; Meilă, 2007). However, there are some limitations. NMI is known to have a selection bias that favors clustering with more clusters. This causes highly over-segmented clustering to give inflated similarity scores (Amelio & Pizzuti, 2015). In addition, NMI is sensitive to cluster size and may generate unintuitive similarity values when one number of clusters far outweighs another in a clustering (McDaid et al., 2011). To overcome this, Adjusted Mutual Information (AMI), Scaled NMI (FNMI), and NMImax have been proposed (Vinh et al., 2010; Amelio & Pizzuti, 2015; McDaid et al., 2011). NMImax normalizes mutual information by the maximum entropy of the two partitions. FNMI reduces selection bias by penalizing the large difference between the number of predicted and reference clusters.

Chapter 3

Synthetic Data

This chapter investigates clustering behaviours in controlled synthetic datasets where the true number of clusters is known. Synthetic datasets give complete control over the number, position, and spread of clusters. They allow for the addition of noise or extra irrelevant features in a controlled manner. This enables the testing of clustering methods without the uncertainties present in real datasets.

Two experiments are conducted. The first experiment examines whether common methods for selecting k in K-Means and GMM, including the elbow method, silhouette score, DBI, CHI, and gap statistic, can recover the correct number of clusters when the clusters are well-separated. The second uses a cube-vertex dataset to examine the effect of irrelevant dimensions and measurement noise on clustering results. Parameters such as the number of noise dimensions and the spread of clusters are adjusted to assess the robustness of various clustering algorithms. Both internal and external metrics are used, such as the silhouette score and the ARI. In the second experiment, only the elbow method is applied to check k because its detection ability was already confirmed in the first experiment.

The findings of these synthetic experiments provide a benchmark for interpreting the real-world datasets in later chapters. The Spotify dataset in Chapter 4 and the MNIST dataset in Chapter 5 contain more complex and noisy conditions. The insights gained from the synthetic experiments guide the choice of methods and the interpretation of results in those chapters.

3.1 Experiment 1: Optimal k Detection with Gaussian Blobs

This experiment is designed to verify whether common k-selection methods can recover the known number of clusters in a low-dimensional setting under controlled conditions. Synthetic datasets with different numbers of Gaussian blobs are generated for the experiment using the `make_blobs` function. Each dataset contained $n = 1000$ points with equal distribution across predefined numbers of clusters. Each cluster is normally distributed with the same spread, removing real-world ambiguities. The k-selection methods for this experiment are the elbow method, silhouette analysis, DBI, and gap statistics.

Two scenarios were evaluated. The first begins with the low-complexity situation, where the number of clusters is fewer than 5, followed by the high-complexity case, with more than 10 clusters. The latter scenario is intended to simulate a dataset like the MNIST, discussed in Chapter 5, with 10 different classes present. For the $k = 3$ dataset, all four k-selection methods correctly identified $k = 3$. This outcome was expected, as the clusters are not overcrowded with minimal overlap.

In the case of $k = 4$, the Elbow Method was the only method that failed to identify the correct number of clusters. This is due to the sensitivity of the rate of change in inertia, where the automated `KneeLocator` implementation selected $k = 3$. The slope reduction between $k = 3$ and $k = 4$ was too gradual to produce a distinct inflection point. However, visual inspection of the elbow plot, which is shown in Figure 3.1, suggested that $k = 4$, indicating that `KneeLocator` may miss subtle but meaningful changes.

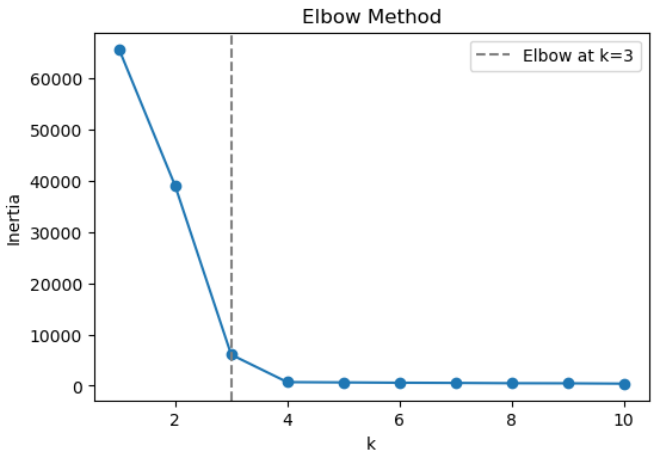


Figure 3.1: Elbow plot for synthetic dataset with $k = 4$ clusters.

When the data contained $k = 10$ clusters, overlapping clusters were observed, as shown in Figure 3.2. The results obtained through the methods differed. The Elbow Method suggested $k = 4$, the Silhouette Score suggested $k = 8$, the Davies–Bouldin Index suggested $k = 7$, and the Gap Statistic suggested $k = 10$, which correctly identified the optimal k. The increased proximity among clusters led each method to emphasise different trade-offs between within-cluster compactness and between-cluster separation. This highlights the difficulty when identifying the number of clusters in high-complexity data with partial overlaps.

In general, Experiment 1 demonstrates that these k-selection methods remain effective when separability reaches a desired statistic but degrade when clusters become indistinguishable. When inertia decreases smoothly, automated techniques like `KneeLocator` can conservatively underestimate k. In practice, especially when dealing with complex or noisy data, a combination of multiple metrics and visual inspection should be used to enhance the robustness of determining the number of clusters.

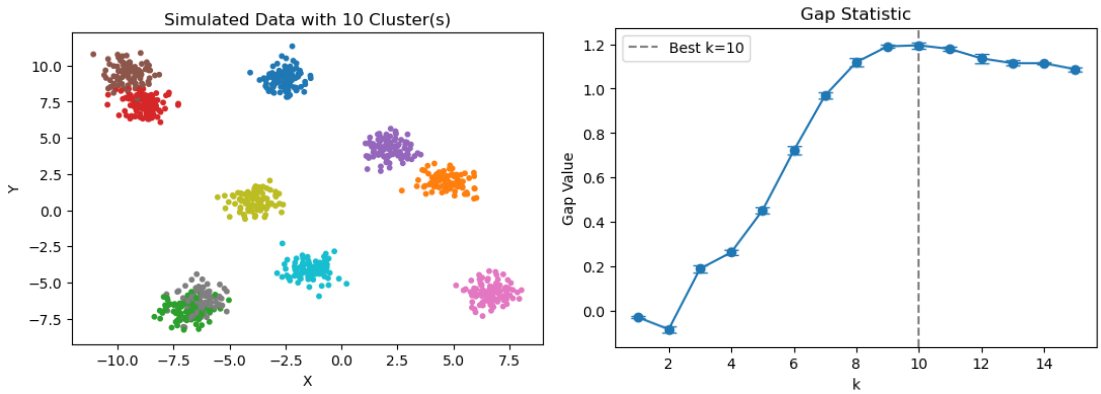


Figure 3.2: Synthetic dataset with $k = 10$ clusters (left), and the corresponding Gap Statistic plot (right), which identifies $k = 10$ as the optimal number of clusters.

3.2 Experiment 2: Effect of Noise on Clustering

This experiment examines the robustness of the clustering when the data contains extra dimensions and measurement noise. At first glance, eight Gaussian-distributed clusters (blobs) were generated, each centered at each vertex of a cube in three-dimensional space, where $(x, y, z) \in \{0, 5\}^3$. Each cluster contains $n = 100$ with the cluster standard deviation of 0.5. Two types of noise settings are applied to this data.

1. Extra noise dimension

   For $d \in \{0, \ldots, 10\}$, append d irrelevant noise features of dimension to X.

   $$X_d \;=\; [\,X \parallel Z_d\,], \qquad Z_d \sim \mathcal{N}(0,1)^{n\times d},$$

   where, X is the original $n \times 3$ data matrix, Z_d is an $n \times d$ matrix of independent Gaussian noise with mean 0 and variance 1, and $[X \parallel Z_d]$ denotes concatenating Z_d as additional columns to X.

2. Within dimension noise

   Add Gaussian noise with standard deviation $\sigma \in \{0.0, 0.05, 0.1, 0.2, 0.5, 0.8, 1.0, 2.0, 3.0\}$ to each original coordinate.

   $$X(\sigma) \;=\; X + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2)^{n\times 3},$$

   where, ε is the Gaussian noise matrix added element-wise to X, and $X(\sigma)$ is the resulting noisy dataset.

Extra dimensions increase dataset dimensionality without adding meaningful structure. Within-dimension noise reduces cluster separation by increasing overlap between points.

In both experimental settings, K-Means, DBSCAN, Agglomerative Clustering, and GMM were fitted to evaluate how these different clustering approaches are influenced in a controlled

noise situation. Both models, K-Means and GMM, are implemented with the fixed number of clusters $k = 8$, matching the generated blobs. The choice of the number of clusters was validated using the elbow method. For DBSCAN, two sets of parameters were tested. The initial setting is set to the constant parameters of $\varepsilon = 0.95$ and `min_samples` $= 5$ using the k-distance graph, which serves as a baseline to compare with the optimal parameters. The second used a grid search to explore within a specific range of the parameters ε and `min_samples`, and select the combination that maximised the silhouette score, with a total of at least 5 clusters. This setting serves as the best parameters for DBSCAN. Lastly, Agglomerative Clustering is run under diverse linkage criteria. Ward's linkage appeared to be the most stable choice among the criteria, creating compact and well-separated clusters even at moderate noise conditions. Contrarily, single linkage was the poorest, which can be explained by the chaining effect, where a few in-between points might result in inappropriate merging of clusters (Xu & Wunsch, 2008).

Stability analysis is conducted to further assess the robustness beyond single execution. Each clustering method is executed twenty times under each noise configuration with a different random seed as one loop. The mean and standard deviation of the silhouette score and ARI are then calculated to describe uncertainty between the runs. Only noise configurations for which at least ten valid runs occurred are considered, ensuring the results are reliable. This viewpoint on stability is critical, as an algorithm that seem competitive under one run may be variable when the data realisation. Algorithms that perform stably across multiple runs are more likely to prove their reliability and consistency. In most cases, such stability is more important than the best single-run result.

Figure 3.3 shows the mean values with shaded areas representing one standard deviation over twenty runs. The first row corresponds to additional noise dimensions, while the second row illustrates the within-dimension noise. As noise dimensions increase, the silhouette score declines for all methods. This reflects the loss of cluster compactness as irrelevant features are added. However, K-Means, Agglomerative, and GMM maintain relatively high ARI scores, due to the fixed number of clusters preserving alignment with true labels despite distance being blurred by extra dimensions. In contrast, DBSCAN fails in high-dimensional spaces. For the fixed-parameters model, the silhouette score is undefined, and ARI collapsed after adding three noise dimensions. Although the best-parameters model performed the best in silhouette metrics, it consistently performed the poorest in ARI. This is due to the density assumption breaking down, where the additional dimensions are defined as noise, making the clusters fragmented.

With the increase of cluster spread, both metrics consistently decreased across all methods. This is expected since the increase in overlapping points diminished the cluster structure, making correct assignment difficult. Interestingly, the DBSCAN best-parameters model maintained high silhouette values, even though ARI scores are the worst among the methods. This happens because parameter tuning allows algorithms to recover compact local groupings without having accurate label recovery. For the DBSCAN fixed-parameters model, the silhouette score shows a slight rise at high spread after declining with moderate spread. This occurred because of the

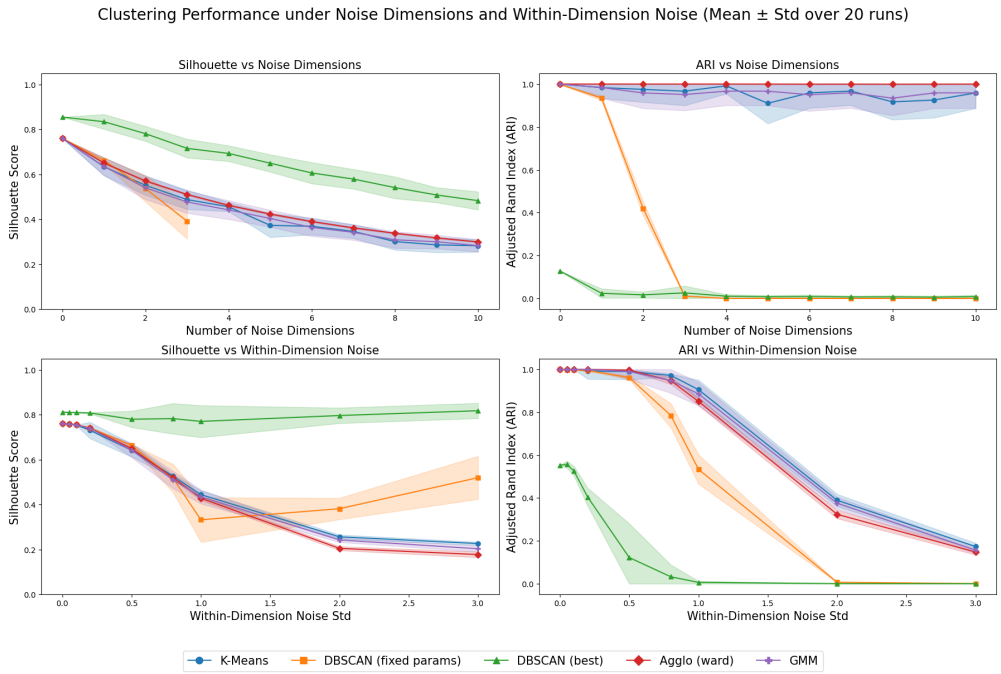


Figure 3.3: Clustering results of K-Means, DBSCAN (fixed and best parameters), Agglomerative (Ward), and GMM with the addition of noise dimensions (top row) and within-dimension noise (bottom row). The performance is measured by Silhouette Score (left column) and Adjusted Rand Index (right column). The results are given as mean ± SD in 20 runs.

predefined parameters no longer match the true density structure. The predefined parameters are still suitable for low spread, able to form clusters. For a higher spread, small residual clusters are formed which are no longer meaningful and not aligned with the ground truth. Nevertheless, K-Means, GMM, and agglomerative performed better, degraded more smoothly than DBSCAN, resulting from the fixed number of clusters enforcing a partition even under noise.

This approach demonstrates that DBSCAN is sensitive to parameter selection. A small adjustment in the parameters can affect the clustering results. A small ε may cause excessive noise labelling, while a large ε may merge distinct clusters. Tuned parameters could recover well-formed, core-like dense regions in some noise scenarios, whereas fixed parameters often yielded excessive noise labelling or broken clusters. This highlights that density-based approaches will often have to be tuned adaptively on high-dimensional or noisy data, but centroid-based and model-based methods will tend to maintain stable behaviour without the need to re-optimize model parameters.

Chapter 4

Spotify Data

The previous chapter presents a controlled baseline to measure the performance of clustering algorithms using synthetic datasets in terms of optimal cluster detection, robustness to noise, and sensitivity to dimensionality. Although such experiments are effective and manageable in interpreting the behaviour of the algorithms, the complexity of real-world data is difficult to capture. For this reason, this chapter focuses on a real-world dataset based on Spotify. The Spotify dataset presents a more complex domain, where the music tracks are characterized by a set of audio features and the intrinsic ambiguity of grouping.

4.1 Data Description

The Spotify dataset used in this study was sourced from Kaggle, which is a publicly available dataset (Hamidani, 2019). It was developed based on the Spotify Web API to provide coherence in the description of track attributes. This dataset comprises 232,725 tracks, each of which encompasses an assembly of audio features and metadata, spanning a diverse variety of musical styles and genres. The metadata includes track name, artist, track ID, and musical identifiers like key, mode, and time signature. Besides these, the dataset includes an abundance of numerical audio features, such as popularity, danceability, energy, loudness, tempo, valence, acousticness, instrumentalness, liveness, and speechiness. These features are popular in acoustic-related music information retrieval research. In combination, these attributes can build a high-dimensional representation of the tracks and provide a compatible foundation to conduct clustering experiments to discover patterns and similarities among the tracks.

4.2 Data Preprocessing

Several preprocessing steps were applied to the Spotify dataset before implementing the clustering algorithms. Since the dataset consists of both categorical and numerical attributes, the filtering process is carried out to select the attributes related to the acoustic and perceptual properties of music. Other non-numeric fields, as well as categorical attributes in musical identifiers,

were filtered out. The remaining features used for clustering are popularity, danceability, energy, loudness, tempo, valence, acousticness, instrumentalness, liveness, and speechiness, which provide a high-dimensional representation of the tracks.

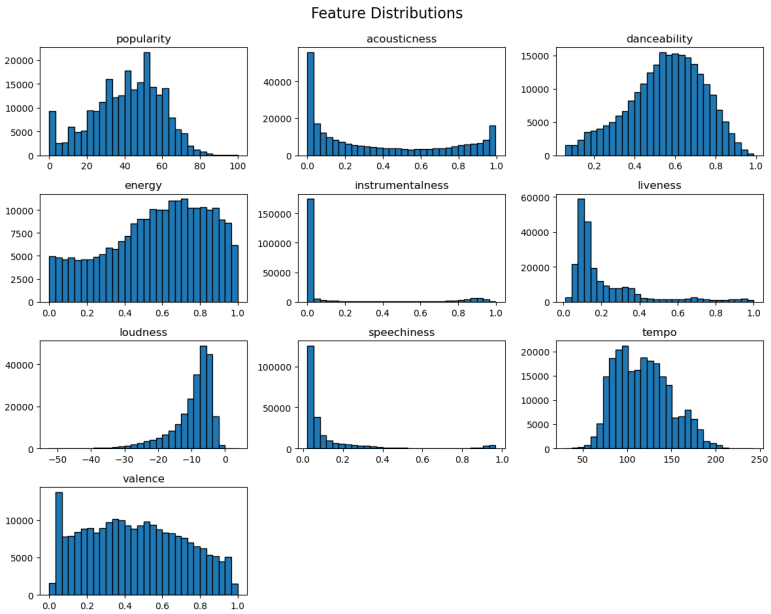


Figure 4.1: Distributions of numerical audio features in the Spotify dataset.

The distribution of the features shown in Figure 4.1 reveals that energy and valence are uniform, popularity, danceability, and tempo follow a bell-shaped pattern, whereas the others are skewed. The skewness and outliers are not removed using any transformation because distance is the major concern. However, standardisation of features is done to facilitate comparison across scales. Furthermore, the correlation analysis shown in Figure 4.2 illustrates a strong relationship between energy and loudness. Such redundancies prompt the application of PCA in clustering. As indicated in Figure 4.3, the first 7 principal components capture approximately 95% of the data variance. This stipulates that much information from the original features can be preserved in a smaller feature dimension, which enhances computational capability. Due to the high memory and computation intensity of agglomerative clustering, the sample size was subsampled to 10,000 tracks to ensure deployable execution, while other methods remain using the full dataset.

4.3 Clustering Algorithms Applied

After preprocessing, a range of clustering algorithms are applied to the Spotify data to investigate how various techniques deal with high-dimensional musical features. These algorithms are K-Means, DBSCAN, agglomerative clustering, and GMM. The selection of algorithms is guided by their theoretical approaches and by the results of the synthetic experiments.

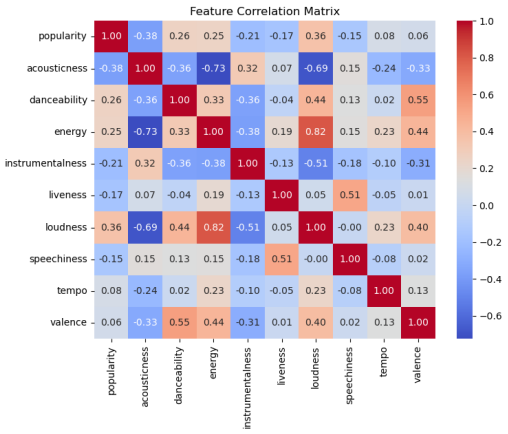


Figure 4.2: Correlation matrix of numerical audio features in the Spotify dataset.

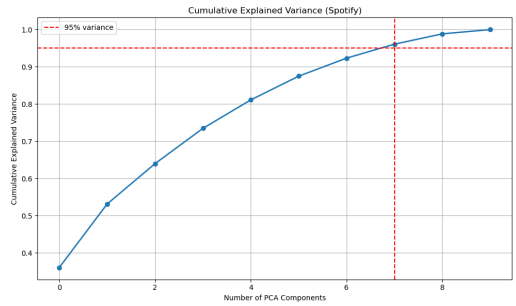


Figure 4.3: Explained variance of principal components for the Spotify dataset, showing 95% of the data variance is around 7 components.

4.3.1 Determining the Number of Clusters in K-Means and GMM

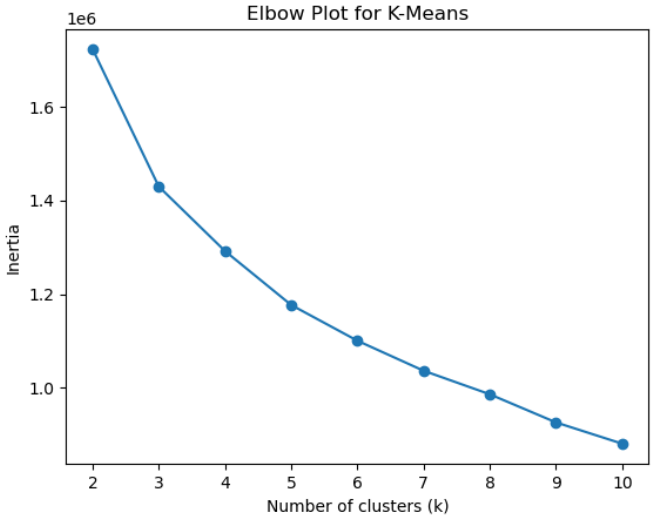


Figure 4.4: Elbow method for K-Means on the Spotify dataset.

To determine the appropriate number of clusters, k, for K-Means and GMM, the elbow method, silhouette analysis, DBI, and gap statistics are examined. The elbow plot shown in Figure 4.4 implies that the appropriate range of k should be somewhere between 3 and 5, since no marginal improvement occurs after that. Internal validity indices support this observation, which is illustrated in Table 4.1. The silhouette score peaks at $k = 3$, and the DBI is also minimal at the same point, indicating compact and well-separated clusters. On the other hand, the gap statistic continues to rise with larger k, and it peaks at $k = 10$, since the stopping rule for selecting an earlier k is not met. For GMM, both AIC and BIC are evaluated, but ambiguous results are observed. Although such differences highlight the sensitivity of the various validity measures, the overall evidence points heavily towards $k = 3$ as the most appropriate for the

Spotify dataset.

Table 4.1: Cluster validity indices for K-Means on the Spotify dataset

k	**Silhouette**	**Davies–Bouldin**	**Gap**	$Gap(k+1) - s_{k+1}$
2	0.312	1.385	0.923	1.029
3	**0.345**	**1.118**	1.017	1.092
4	0.196	1.613	1.042	1.126
5	0.172	1.688	1.094	1.129
6	0.158	1.681	1.129	1.180
7	0.165	1.639	1.158	1.183
8	0.163	1.608	1.180	1.246
9	0.170	1.529	1.219	1.246
10	0.166	1.538	**1.248**	—

4.3.2 Parameter Tuning in DBSCAN

DBSCAN is tested out on the Spotify dataset to determine whether clusters of different densities and outliers can be identified and separated. The k-distance plot is applied to tune the parameter ε and `minPts`. Figure 4.5 demonstrates an example of a k-distance plot using full-dimensional data with `minPts` = 20. This plot indicates the difficulty in estimating ε, where there is no distinct point of separation. To address this, a range of ε and `minPts` are experimented over multiple dimensions of the data, including full feature and PCA-reduced ones.

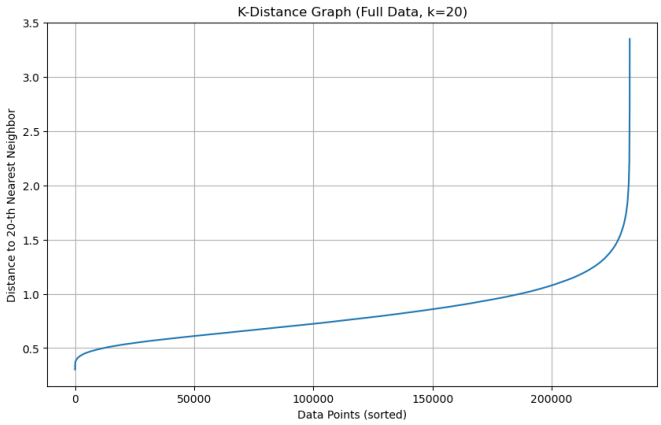


Figure 4.5: k-distance plot for the Spotify dataset with minPts = 20.

Table 4.2 presents some of the results of DBSCAN on full-dimensional data. Different sets of parameter settings yield very different results. A small value of ε and `minPts` give hundreds of small clusters, while a large setting produces only a few large clusters. However, the parameters of $\varepsilon = 1.2$ and `minPts` $= 20$ have the best results, which return three clusters with a positive silhouette score of 0.301, DBI score of 0.806, and CHI score of 16228.50, as well as the lowest noise percentage. Besides, DBSCAN is also used on 2-dimensional and 7-dimensional data, illustrated in Tables 4.3 and 4.4. The 2-dimensional data perform the worst,

producing thousands of clusters with negative silhouette scores, large DBI values, and small CHI values for each combination. For 7-dimensional data, the results are better than 2-dimensional data, but still modest compared to full-dimensional data, producing fewer and more interpretable clusters. For example, $\varepsilon = 0.8$ with `minPts` = 14 create seven clusters with a small positive silhouette of 0.086. In general, such behaviour illustrates the sensitivity of DBSCAN to the choice of parameters. Though DBSCAN manages to solve the problem of meaningful clusters, K-Means and GMM are more suitable.

Table 4.2: DBSCAN results on full-dimensional Spotify data

ε	**minPts**	**Clusters**	**Noise (%)**	**Silhouette**	**DBI**	**CHI**
0.8	5	539	13.00	-0.514	1.013	66.52
1.0	10	19	5.60	-0.106	1.017	1809.72
0.8	15	29	22.39	-0.294	1.116	1025.45
0.9	20	11	14.58	-0.241	1.233	3070.18
1.0	20	3	8.15	**0.387**	**0.737**	15972.18
1.2	20	3	2.48	0.301	0.806	**16228.50**

Table 4.3: DBSCAN results on 2D PCA-reduced Spotify data

ε	**minPts**	**Clusters**	**Noise (%)**	**Silhouette**	**DBI**	**CHI**
0.010	4	6670	25.54	-0.453	6.966	17.66
0.020	4	2058	7.28	-0.433	**6.250**	22.99
0.012	5	4131	23.91	-0.411	7.462	24.33
0.015	5	3032	16.11	-0.399	7.493	38.65
0.010	6	3630	36.61	-0.400	7.856	16.95
0.018	6	1746	14.01	**-0.357**	7.435	**63.14**

Table 4.4: DBSCAN results on 7D PCA-reduced Spotify data

ε	**minPts**	**Clusters**	**Noise (%)**	**Silhouette**	**DBI**	**CHI**
0.55	12	99	23.69	-0.458	1.095	316.89
0.65	12	55	12.07	-0.364	1.027	636.56
0.70	14	17	9.35	-0.200	0.983	2154.91
0.80	14	7	4.52	**0.086**	0.982	5821.26
0.75	16	11	7.14	-0.083	0.919	3467.09
0.80	16	6	4.94	0.053	**0.783**	**6960.90**

4.3.3 Linkage Selection in Agglomerative Clustering

The Spotify data is clustered using agglomerative clustering to identify whether the hierarchical structure of the data will help to identify meaningful clusters of tracks. Different types of linkage criteria are compared, such as Ward's, complete, average, and single linkage, as shown in Figure 4.6. The dendrograms illustrate that different linkage options produce various cluster formation patterns.

Ward's linkage presents the best-balanced clusters, having relatively compact clusters and deeper separation of the branches, resulting in more interpretable clusters compared to others.

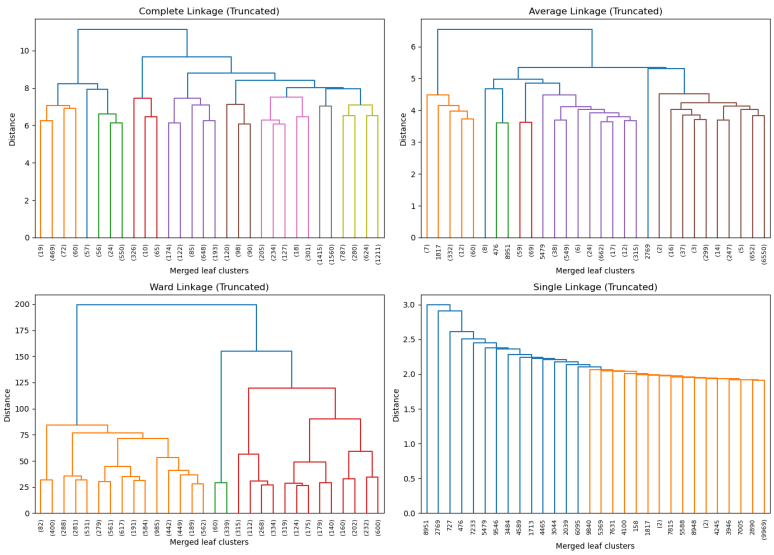


Figure 4.6: Agglomerative clustering dendrograms using complete (top left), average (top right), Ward's (bottom left), and single (bottom right)linkage methods.

The complete and average linkage produce moderately coherent structures, but there are less pronounced separations at higher hierarchical levels. By contrast, single linkage has the chaining effect, in which data points are connected in chains, resulting in poor clusters. All in all, Ward's linkage appears to be the most appropriate, since it reduced the within-cluster variance and produced a clearer hierarchy, the dendrogram suggesting that the dataset can be separated into three clusters.

4.3.4 Covariance Structures in Gaussian Mixture Models

Different GMM covariance structures are implemented to investigate the effect of each covariance on clustering evaluation results in the Spotify dataset. The evaluation includes full, tied, diagonal, and spherical structures, and compares across cluster component numbers ranging from 2 to 10 using different metrics. Figure 4.7 summarizes the results in the form of heatmaps of Silhouette score, DBI, and CHI.

Spherical covariance appears to provide the most promising performance, achieving optimal results on all metrics with the highest silhouette score, lowest DBI, and largest CHI at $k = 3$. This shows that spherical covariance provides compact and well-separated clusters across cluster components. The tied covariance also performs relatively well, with competitive metric values, indicating the capability to capture some meaningful structure of the data, though weaker than spherical. In comparison, the full covariance cannot obtain stable results, frequently showing low or negative estimated silhouette scores, as well as a large DBI value and a low CHI value. Diagonal covariance performs marginally better than full covariance but poorly compared to spherical and tied covariances. Notably, the benefit of the spherical covariance can be justified by the relatively weak correlations of Spotify attributes. The full covariance model has a more

complex structure and is prone to overfitting the data. Further, the spherical covariance is the most computationally feasible, taking significantly less time to fit than the others. In general, the best-fit covariance structure for GMM is spherical covariance, and three cluster components show the best results across all metrics, which agrees with the findings in Section 4.3.1, where $k = 3$ is identified as the most suitable choice for both K-Means and GMM.

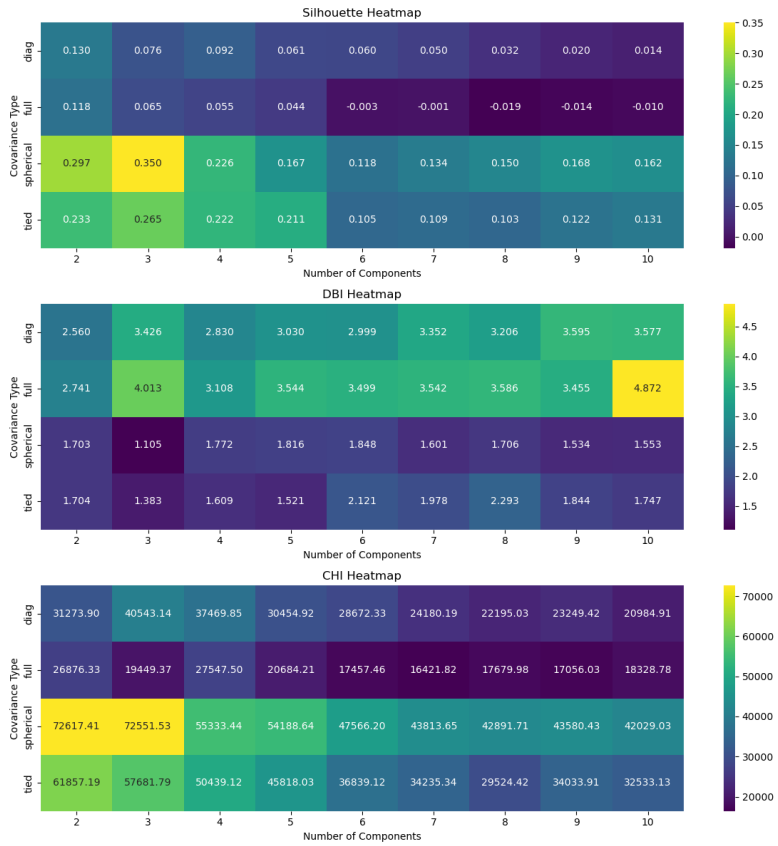


Figure 4.7: Heatmaps of Silhouette scores, DBI (top), and CHI (middle) for GMM (bottom) under different covariance structures and cluster components.

4.4 Comparative Analysis

Comparison of the four clustering algorithms is performed on the selected feature dimensions, including 2-, 7-, and full-dimensional data. The internal metrics evaluated are the silhouette score, DBI, and CHI, and the results are summarized in Table 4.5. The best performance in K-Means is by using 2-dimensional data, which attains the best score for each metric with a silhouette score of 0.600, DBI of 0.554, and CHI of 292538. At higher dimensions, each metric continues to diminish, indicating weaker compactness and clusters separation. The same trend is observed with GMM, which performs similarly to K-Means, with only slight differences, making it difficult to point out a winner. For agglomerative clustering, the results observed

are likewise, with the 2-dimensional data having the best silhouette and DBI value. However, the CHI score is significantly smaller since the resampled dataset was used. Nevertheless, the cumulative evidence suggests that all three algorithms yield relatively similar results, with low-dimensional representations showing superior performance. On the other hand, DBSCAN behaves differently, where the 2-dimensional data fragments into thousands of small clusters with poor metrics. However, it identify three clusters, with moderate results for the metrics, reflecting that DBSCAN can uncover meaningful structure with the appropriate tuning.

Table 4.5: Comparative clustering performance on Spotify across selected PCA dimensions.

Algorithm	Dimension	Silhouette	DBI	CHI
K-Means	2D PCA	**0.600**	**0.554**	**292538**
	7D PCA	0.369	1.041	83046
	Full	0.345	1.118	73092
Agglomerative (Ward)	2D PCA	**0.603**	**0.493**	**9908**
	7D PCA	0.401	0.911	3271
	Full	0.266	1.410	2405
GMM (Spherical)	2D PCA	**0.601**	**0.541**	**289119**
	7D PCA	0.393	0.971	89043
	Full	0.350	1.105	72552
DBSCAN†	2D PCA	-0.433	6.250	23
	7D PCA	0.086	0.982	5821
	Full	**0.387**	**0.737**	**15972**

†DBSCAN values are shown for representative parameter settings (best observed per dimension).

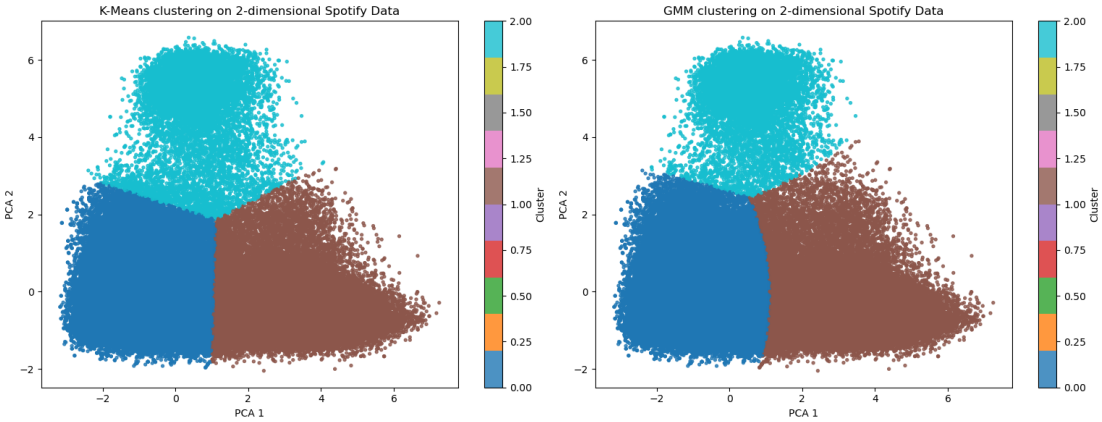


Figure 4.8: Clustering results on 2-dimensional Spotify data on K-Means (left) and GMM (right).

Besides the internal measurements, the clustering outputs are visually assessed. Figure 4.8 shows the separation of the clusters in a 2-dimensional PCA space, and the results are almost identical for K-Means and GMM, proving consistency in the evaluation metrics as well. In comparison, DBSCAN has a different pattern, where one large cluster dominates, while a substantial proportion of samples are classified as noise, shown in Figure 4.9.

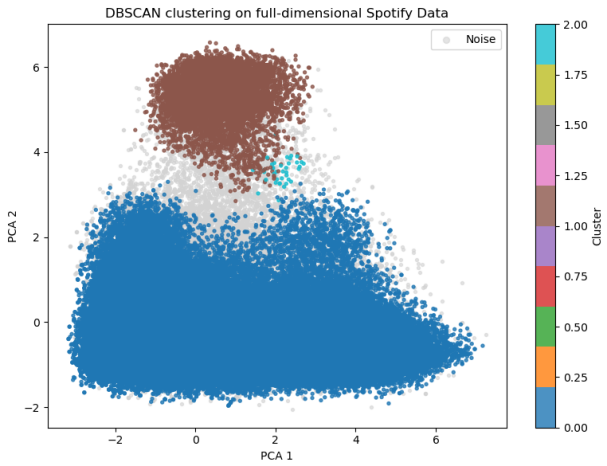


Figure 4.9: DBSCAN clustering result on the full-dimensional Spotify data, where the grey points represent noise.

To enhance understanding of the cluster results, the average values of the musical features within each cluster are compared. Figure 4.10 presents the normalized cluster profiles of the K-Means. Cluster 0 is defined by high-valence and tempos as well as popular and energetic danceable tracks, suggesting mainstream pop or dance music. Cluster 1 is characterized by high acousticness and instrumentalness, but low energy and danceability, indicating a quieter style of acoustic or classical. Cluster 2 is distinguished by high liveness, speechiness, and low popularity, pointing towards live recordings, rap, or hip-hop tracks.

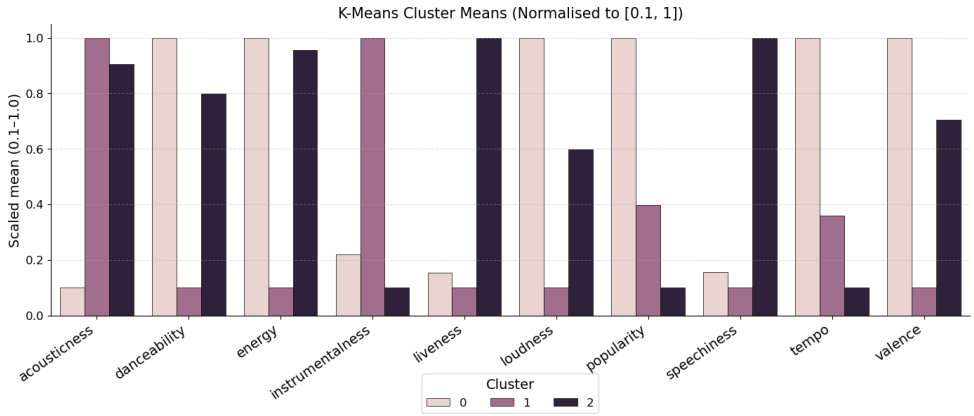


Figure 4.10: Cluster feature means for K-Means on the Spotify dataset, normalised to [0.1,1].

Comparative analysis reveals that dimensionality reduction using PCA improves the quality of clusters with 2-dimensional data, resulting in the most coherent and interpretable clusters among the algorithms. While K-Means and GMM provide similar results, DBSCAN demanded some careful tuning, and agglomerative produced a competitive outcome. Importantly, the clusters reflect intuitive categories, emphasizing that the algorithm not only provides strong internal metrics, but also interpretable song-like patterns in the Spotify dataset.

Chapter 5

MNIST Data

This chapter examines the MNIST dataset of handwritten digits by applying various clustering approaches. The MNIST dataset provides a large amount of handwritten digit images, each with a corresponding digit label ranging from 0 to 9. These labels serve as ground truth, providing another perspective for evaluation and enabling a more rigorous assessment of the clustering performance. The chapter begins with the description of the dataset and the preprocessing steps undertaken to prepare for clustering. This is followed by a brief description of the clustering algorithms employed, along with the interpretation of the clustering results.

5.1 Data Description

The MNIST dataset is commonly employed as a benchmark in machine learning and pattern recognition research (LeCun et al., 1998). It consists of 70,000 grayscale images of handwritten digits from 0 to 9. The resolution of each image is 28×28 pixels, resulting in 784 features per sample when flattened into a vector. The digit labels represent the ground truth, indicating the actual digit class of each image. This provides an assessment of clustering performance through external validation measures. The dataset used in this study is obtained from the scikit-learn built-in dataset, which is already in the form of a 784-dimensional feature vector for each image.

This dataset can be characterized as high-dimensional data, and the disparity in handwriting styles constitutes a challenging clustering task. For example, digits such as 4 and 9, as well as 3 and 8, are often similar and occur in overlapping regions of the feature space. Figure 5.1 shows examples of handwritten digit images from the MNIST dataset.

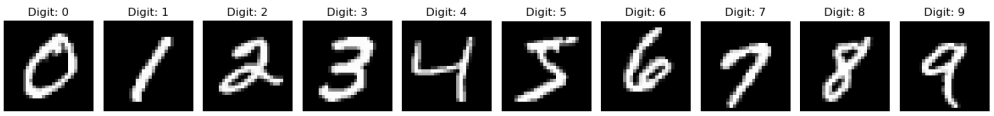


Figure 5.1: Examples of handwritten digit images with digit labels from the MNIST dataset.

5.2 Data Preprocessing

Before applying the clustering algorithms to the MNIST dataset, some preprocessing procedures are conducted. The raw pixel intensity values range from 0 to 255, and are normalized to the interval $[0, 1]$. Normalization is required since clustering algorithms are based on distance estimation, and larger features would dominate the computations. Through normalizing, the influence of each feature can be balanced, thus contributing to a more reliable clustering process. The standardization process is also performed, rescaling the data to zero mean and unit standard deviation. Nevertheless, no improvement is obtained in terms of clustering performance, so the analyses in this study are conducted only on normalized data. In addition, the MNIST dataset is resampled to 10,000 observations to apply agglomerative clustering, as the full dataset size led to an exorbitant memory consumption. With such resampling, the algorithm is able to be applied efficiently and retains a representative sample of the data.

Since the dimensionality of the MNIST dataset is very high, PCA is applied to reduce the dimensionality while retaining most of the variance. The cumulative explained variance plot shown in Figure 5.2 is analysed to examine the number of components that captured the most information in the dataset. As shown in the figure, 150 components retain approximately 95% of the variance. Instead of setting a single threshold, clustering is carried out on the entire feature space as well as on multiple PCA-reduced datasets. This allows for comparison of clustering behaviours under different feature settings.

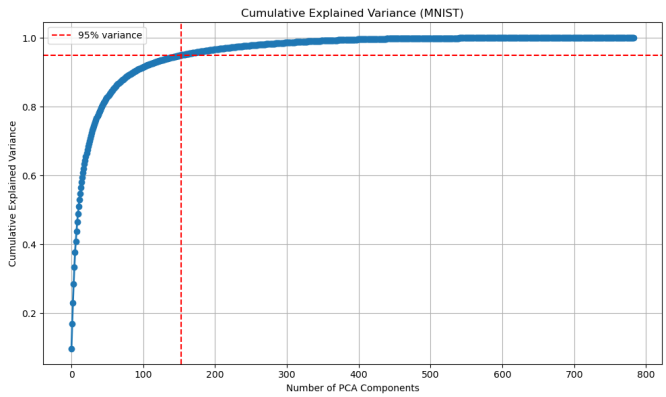


Figure 5.2: Cumulative explained variance for PCA components on the MNIST dataset, showing that 95% of the data variance is around 150 components.

5.3 Clustering Algorithms Applied

Four clustering algorithms are implemented on the MNIST dataset, giving a performance comparison study. This section describes the considerations and trade-offs for each algorithm when selecting the number of clusters, parameter tuning, and model configurations.

5.3.1 Determining the Number of Clusters in K-Means and GMM

The selection of the number of clusters directly affects both the performance of K-Means and GMM. As there are ten-digit classes in the MNIST dataset, the initial choice of $k = 10$ is considered. To validate this assumption, the elbow method and silhouette analysis are conducted in a range of $k = 2$ to 20, using different data dimensions.

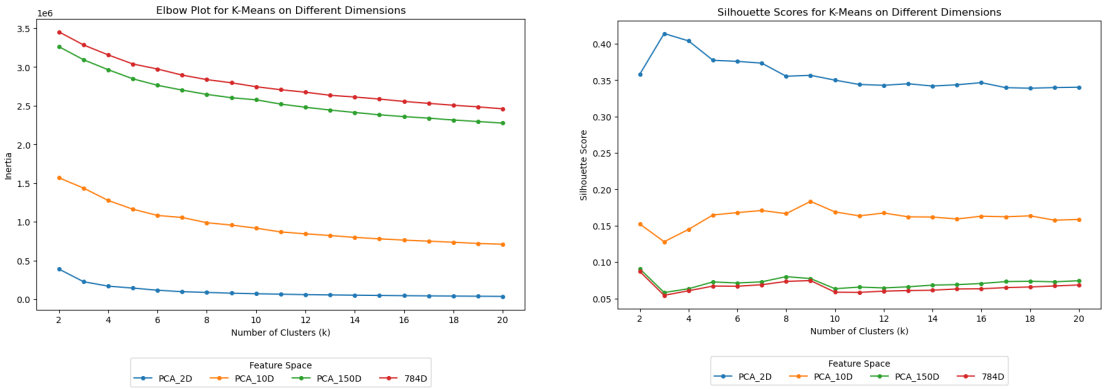


Figure 5.3: Elbow method (left) and Silhouette scores (right) for K-Means across different dimensions.

Based on the results shown in Figure 5.3, the two-dimensional PCA data have the highest silhouette score and the lowest inertia values. This indicates that the data is well-separated, although the distances are compressed in the low-dimensional data and do not preserve the full variance of the dataset. In contrast, the full 784-dimensional and 150-dimensional PCA data yield the lowest silhouette score and the highest inertia value. This reveals that clustering suffers when dealing with high-dimensional data. Whereas the 10-dimensional PCA data produces a more balanced result with a combination of proportionate separation and compactness.

Across all dimensions, the inertia curves decrease monotonically with increasing k, without a sharp elbow point in the elbow plot. Similarly to the silhouette analysis, there is a slight peak around $k = 9$. However, the scores at $k = 8$ and $k = 10$ are similar, with only a marginal difference. This reflects the inherent difficulty of choosing the optimal number of clusters in overlapping digit classes and blurred cluster boundaries. Despite this, K-Means and GMM are set at a value $k = 10$ based on prior knowledge of the dataset architecture.

5.3.2 Parameter Tuning in DBSCAN

DBSCAN requires two parameters to form a cluster, the neighbourhood radius ε and the minimum number of points `minPts`. The clustering performance significantly depends on the parameters selected, especially for high-dimensional datasets. To estimate the suitable value of ε, the k-distance plot needs to be employed, with experimentation using different `minPts`. This plot is measured based on the distances to the k-th nearest neighbour, sorted in ascending order. The point of maximum curvature in the plot gives a heuristic estimate of the suitable

ε, yet in practice, this point can be difficult to define, as shown in Section 4.3.2 (Ester et al., 1996). Various `minPts` values are tested across different-dimensional data, and the results are summarized in Tables 5.1 – 5.3.

Table 5.1: DBSCAN results on full 784-dimensional MNIST data

ε	**minPts**	**Clusters**	**Noise Points**	**Noise (%)**	**ARI**	**NMI**	**Silhouette**
5	5	10	64855	92.65	0.0198	0.1479	-0.161
15	20	3	27247	38.92	0.0370	0.0746	0.287
20	20	2	9841	14.06	0.0067	0.0317	0.277
25	20	2	3792	5.42	0.0008	0.0117	**0.428**
10	50	5	62279	88.97	**0.0465**	**0.2318**	0.244

Table 5.2: DBSCAN results on 2D PCA-reduced MNIST data

ε	**minPts**	**Clusters**	**Noise Points**	**Noise (%)**	**ARI**	**NMI**	**Silhouette**
0.028	8	1294	46283	66.12	0.0135	0.1599	0.250
0.040	8	700	18794	26.85	**0.1494**	**0.2613**	-0.414
0.028	10	617	58089	82.98	0.0026	0.1216	**0.368**
0.040	10	643	26954	38.51	0.0997	0.2310	-0.328
0.028	12	233	64877	92.68	0.0005	0.0804	0.441

Table 5.3: DBSCAN results on 150D PCA-reduced MNIST data

ε	**minPts**	**Clusters**	**Noise Points**	**Noise (%)**	**ARI**	**NMI**	**Silhouette**
4.3	100	2	52873	75.53	0.0786	**0.2725**	0.162
4.7	100	2	43247	61.78	**0.1034**	0.2560	**0.215**
4.9	100	3	37100	53.00	0.1134	0.2678	0.069
5.1	125	2	33491	47.84	0.1091	0.2498	0.183
5.3	150	2	29166	41.67	0.1046	0.2474	0.177

The results from the tables reveal that data of different dimensionality requires different parameter values, which made DBSCAN struggle. Data with small dimensions often need smaller parameter values. For the full-dimensional data, it was impractical to set `minPts` values based on the feature space size, so a value of 20 was chosen. At small values of ε, DBSCAN often produced fragmented clusters and high noise rates, and the cluster structure would collapse to form one or two large clusters when ε was rendered larger. The presented parameter combinations in the tables are chosen as the most informative ones, not a full list of the results. Generally, none of the tested settings provides good clustering results, with all settings reporting low ARI, NMI, and Silhouette scores across dimensions.

5.3.3 Linkage Selection in Agglomerative Clustering

The selection of linkage criteria is important in agglomerative clustering, where these parameters represent the measurement of distances between clusters. Four linkage methods are tested,

including Ward's, complete, average, and single linkage. The results are present in Table 5.4. The outcome reveals that Ward's linkage has the optimal clustering performance, with the highest ARI and NMI, and a moderate silhouette score. By comparison, the other linkages perform poorly, with complete linkage generating low validity scores. Average linkage has the best silhouette score but a poor validity index. Single linkage yields almost zero external validation metrics, verifying that it fails to create meaningful groups. Based on the results, Ward's linkage is chosen for the subsequent analyses.

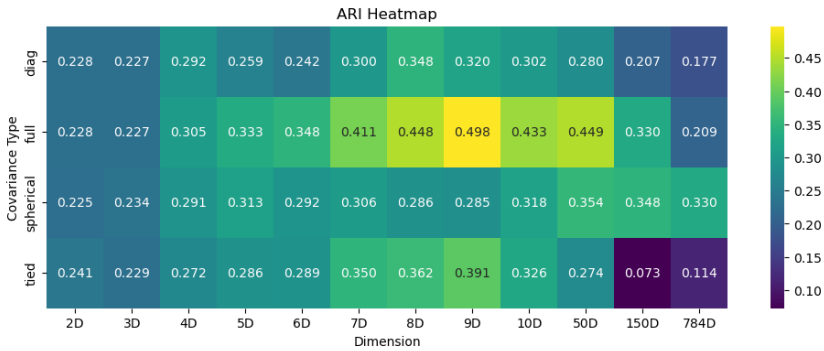
Table 5.4: Agglomerative clustering results with different linkage criteria on MNIST data

Linkage	ARI	NMI	Silhouette
Ward	**0.4705**	**0.6554**	0.0316
Complete	0.1378	0.3343	0.0190
Average	0.0476	0.2426	**0.0617**
Single	0.0000	0.0018	0.0471

5.3.4 Covariance Structures in Gaussian Mixture Models

An important parameter in GMM is the covariance structure that determines the shape and orientation of the Gaussian components. This paper tests four different types of covariances, such as full, tied, diagonal, and spherical covariance. The heatmaps in Figure 5.4 show the summarised results of the ARI, NMI, and silhouette scores.

The analysis illustrates that the full covariance type achieves the highest clustering performance, especially in the 9-dimensional data. It has the highest validity scores and a moderate silhouette score, indicating the ability to model the complex distribution of handwritten digits compared to others. The tied, diagonal, and spherical covariance types have similar moderate results, with the tied covariance form slightly better than the other two. Within any covariance type, the silhouette scores sharply declined in higher-dimensional space, indicating the effect of the dimensionality curse. Overall, these experiments demonstrate that the advantage of choosing the correct type of covariance, with full covariance and moderate PCA reduction, 9-dimensional data provides the best balance between external validation and cluster separation.



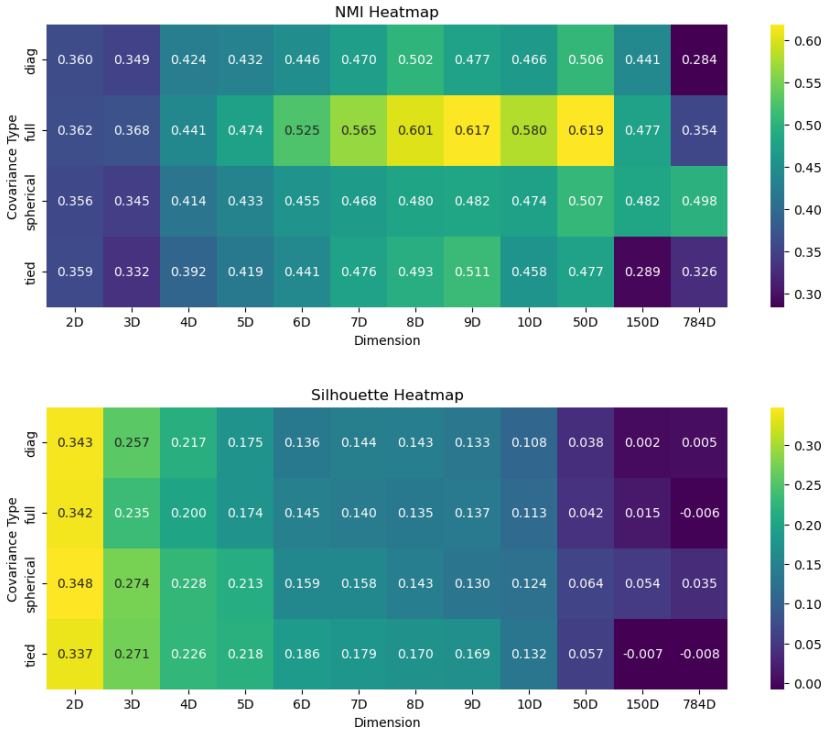


Figure 5.4: Heatmaps of ARI (top), NMI (middle), and Silhouette scores (bottom) for GMM under different covariance structures and data dimensions.

5.4 Comparative Analysis

Table 5.5 displays the comparative results for the clustering algorithms on the MNIST dataset with selected PCA dimensions. For each algorithm, the best configurations are marked in bold. K-Means performed best with the 9-dimensional data, achieving an ARI of 0.406 and NMI of 0.508, along with a reasonable silhouette score. In 2-dimensional data, the maximum silhouette score is observed, corresponding to better cluster separation in highly contracted spaces, albeit with the loss of structural variance. As high dimensionality is observed, agglomerative clustering with Ward linkage had a better result. With 150-dimensional data, the ARI and NMI of 0.496 and 0.674, which surpass the K-Means. Nevertheless, the silhouette scores remained low. GMM with full covariance also performs strongly, yielding the best ARI of 0.498 and NMI of 0.617 at 9 dimensions. These findings indicate that GMM can capture more complex data, like the MNIST, more effectively than K-Means or agglomerative clustering, after moderate PCA reduction. On the other hand, DBSCAN fares badly on the MNIST dataset. Although the parameters are fine-tuned in different dimensions, it still tends to produce only a few clusters and many noise points, shown in Section 5.3.2. This is because estimating parameters for high-dimensional data is difficult for DBSCAN. Overall, the results suggest that clustering in full-dimensional features is less effective, whereas dimensionality reduction significantly accelerates performance while preserving informative structure.

Table 5.5: Comparative clustering performance on MNIST across selected PCA dimensions.

Algorithm	PCA Dimension	ARI	NMI	Silhouette
K-Means	2D PCA	0.248	0.362	**0.351**
	9D PCA	**0.406**	**0.508**	0.180
	10D PCA	0.393	0.501	0.178
	150D PCA	0.367	0.500	0.066
	Full (784D)	0.361	0.491	0.059
Agglomerative (Ward's)	2D PCA	0.249	0.367	**0.307**
	9D PCA	0.428	0.575	0.115
	10D PCA	0.355	0.513	0.116
	150D PCA	**0.496**	**0.674**	0.036
	Full (784D)	0.471	0.655	0.032
GMM (Full)	2D PCA	0.228	0.362	**0.342**
	9D PCA	**0.498**	**0.617**	0.137
	10D PCA	0.433	0.580	0.113
	150D PCA	0.330	0.477	0.015
	Full (784D)	0.209	0.354	-0.006

Along with numerical measures, visualisations provide additional information about the quality of clustering. Figure 5.5 denotes the clustering outcome of GMM with full covariance on 9-dimensional MNIST data that is reduced into 2D space. The clusters of digits are mostly segregated, but overlapping classes can be observed, indicating the complexity of the task.

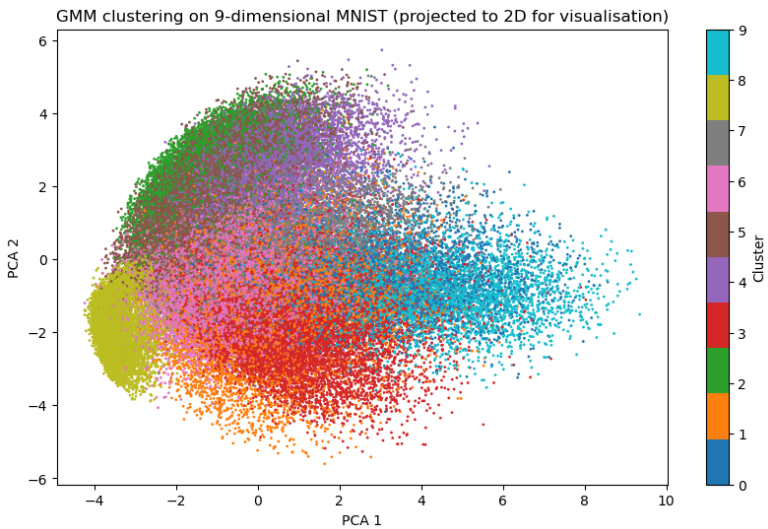


Figure 5.5: GMM clustering on 9-dimensional MNIST projected to 2D for visualisation.

As an additional validation of clustering, the confusion matrix between the true labels of digits and cluster assignments is given in Figure 5.6. While certain clusters are closely aligned with a single digit, such as clusters 1, 2, 7, and 8, others have significant overlap, such as clusters 3, 4, 5, and 6. This demonstrates the difficulty of separating the visually similar digits in the MINIST dataset.

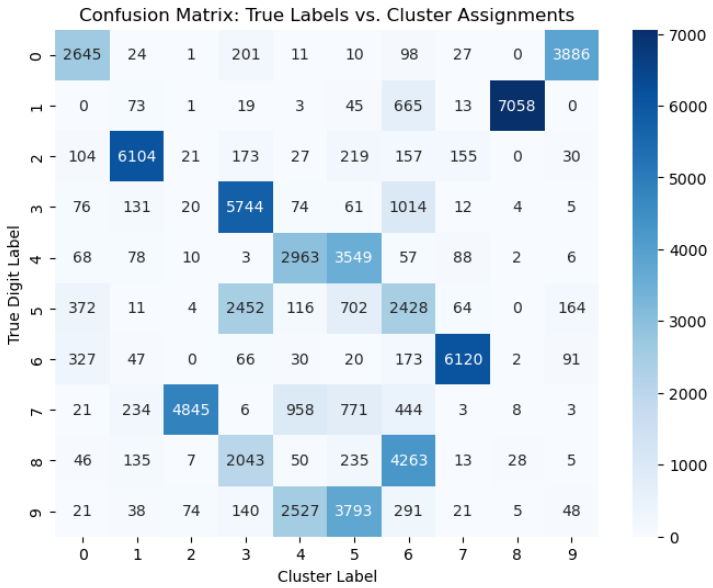


Figure 5.6: Confusion matrix of true digit labels versus GMM cluster labels on 9-dimensional MNIST.

Lastly, examples of images in some clusters can be found in Figures 5.7 and 5.8. There is high homogeneity in cluster 8, as seen in clear digit 1 representations. In contrast, Cluster 6 combines digits 5, 3, and 8, reflecting the uncertainties of distinguishing between similar handwritten digits having a similar skeleton. These visualizations demonstrate the strengths and limitations of GMM, and explain why GMM is the best choice in combination with moderate PCA reduction when clustering the MNIST dataset.

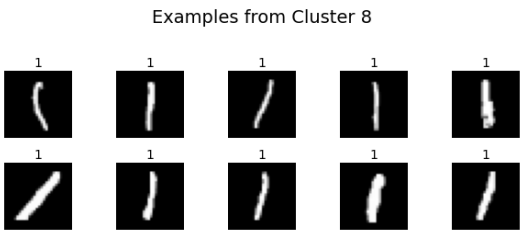


Figure 5.7: Examples of digit images from Cluster 8, showing predominantly digit of "1".

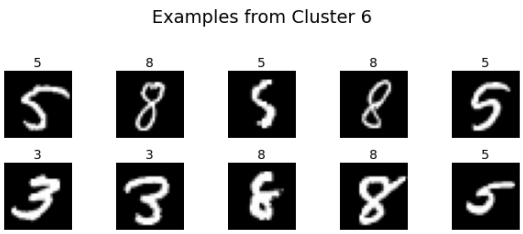


Figure 5.8: Examples of digit images from Cluster 6, showing mixed digits of "5," "3," and "8".

Chapter 6

Discussion

This chapter collates the findings of the three datasets and provides a comparative discussion of the four clustering algorithms in this study. Specifically, the discussion emphasizes the suitability of each algorithm for data with different characteristics such as dimensionality, noise, and shape. In addition, the convergence behaviour and capabilities of algorithms are examined based on the observed performance.

6.1 Influence of Dataset Characteristics

The experiments on the synthetic, Spotify, and MNIST datasets show that clustering performance depends not only on the algorithm itself but also on the nature of the dataset.

Cluster separability

In the synthetic Gaussian blobs, where clusters were well-separated and compact, the algorithms performed relatively well, though with some minor differences. The k-selection techniques were also capable of finding the accurate number of clusters. Yet in the MNIST dataset, where adjacent classes in the digit scale, such as the digits 3 and 8 or 4 and 9, can be overlapped and confused, all the algorithms struggled, with moderate external validation scores at best. This reiterates the fact that clustering works much better when natural group boundaries are clear.

Noise and irrelevant features

The synthetic cube experiment demonstrated the different reactions of the algorithms to noise. DBSCAN was very sensitive, tending to break apart clusters or deem most points as noise when irrelevant dimensions are included. On the contrary, K-Means, Agglomerative, and GMM were more stable, although compactness was decreased over time. These findings suggest that noise immunity is not general and should be taken into consideration when choosing an algorithm.

Volume of data

Another factor that affected the feasibility of the algorithm is the dataset size. The Spotify

dataset, which consists of over 200,000 tracks, needed resampling for agglomerative clustering, whereas it worked well in K-Means and GMM. Similarly, the MNIST dataset required subsampling for Agglomerative clustering. Therefore, in the case where an algorithm is theoretically appropriate, it may not be feasible at any scale due to its computational complexity.

Sparsity

In truly high-dimensional spaces, such as the 784 raw-pixel features of MNIST, density estimation collapsed because of the concentration of distance. DBSCAN effectively considered almost all points as noise, indicating that density-based clustering could not deal effectively with sparse, high-dimensional data.

6.2 Role of Dimensionality Reduction

Dimensionality reduction was determined as an essential stage in situations with Spotify and MNIST. Without it, the cluster quality was poor, as the curse of dimensionality makes distance-based measures less informative. In the case of the Spotify dataset, reducing to 2 components invariably improved compactness and separation, with high internal validation scores. In the MNIST dataset, the best trade-off was achieved by extracting the principal components to 9, where K-Means and GMM returned the highest values of external validation scores. Remarkably, collapsing the MNIST dataset to just 2 dimensions improved the silhouette score but decreased the label alignment, revealing that further dimensionality reduction may lose the data information. These findings reinforce the earlier research that stated the importance of preprocessing for high-dimensional clustering. Nonetheless, the choice of the number of retained components should be a trade-off between interpretability and adherence to the underlying data structure.

6.3 Practical Implications

The findings have several implications for practitioners who may want to deploy clustering techniques in practice.

- K-Means works best on low-dimensional and well-separated data where speed and size matter.
- DBSCAN works well when dealing with spatial data or identifying noise points, but it is unstable in high-dimensional and sparse data environments like MNIST.
- Agglomerative clustering is an intuitive method that gives meaningful hierarchical representations, but was slow to execute and impractical on large datasets.
- GMM is better suited to a complex, overlapping structure, able to capture elliptical clusters than others, but requires careful selection of the covariance structure.

In practice, this implies that the choice of the algorithm should be linked to the characteristics of the dataset. K-Means would consider the most suitable trade-off of accuracy and efficiency when the data is clean and compact. In the case of ambiguous or overlapping clusters, GMM is preferred. Agglomerative clustering is preferable in cases where interpretability is important, whereas the DBSCAN algorithm should be used only in low-dimensional settings where the density structure is meaningful.

6.4 Overview of Algorithm Capabilities and Limitations

Along with the dataset-specific analysis, Table 6.1 illustrates the generalization strengths and weaknesses of the four algorithms under examination in terms of how each performs in shape handling, convergence behaviour, and practical capability. This overview dovetails with the experimental results by explaining why the various algorithms performed well or poorly under various circumstances.

Table 6.1: Comparative Algorithm Performance

Algorithm	**Shape Handling**	**Convergence**	**Capability**
K-Means	Performs well for spherical and convex clusters but struggles with irregular or varying density.	Fast convergence and stable, but sensitive to initialization and outliers.	Scalable to large datasets and works best in low or medium-dimensional data.
DBSCAN	Strong with arbitrary shapes and density-based clusters, but fails in sparse data.	Deterministic once parameters are tuned, but sensitive to parameter selection.	Good for low-dimensional spatial data, ineffective in high-dimensional data.
Agglomerative	Able to capture non-spherical clusters depending on linkage, but sensitive to noise and chaining effects.	Deterministic, with no random initialization required, but computationally heavy.	Suitable for small or medium-sized datasets and provides hierarchy for interpretability.
GMM	Able to handle elliptical and overlapping clusters depending on the covariance structure.	Sensitive to initial parameters, the EM algorithm may converge to local optima.	Effective in medium or high-dimensional data, but computationally more costly depending on the covariance structure.

Chapter 7

Conclusions & Future Work

This chapter concludes the findings of this study and reflects the insights gained from applying clustering algorithms to datasets of different complexities. It wraps up by summarizing the work contributions, limitations, and areas for future research directions.

7.1 Conclusion

This study has explored the comparative performance of four clustering algorithms, namely K-Means, DBSCAN, Agglomerative clustering, and Gaussian Mixture Models (GMM), by applying them to three different datasets, including synthetic data, Spotify track features, and MNIST handwritten digits. This was achieved by following the general goal of observing how clustering algorithms perform across data scenarios, changes in dimensionality, cluster separability, noise level, and scale.

Synthetic experiments served as a controlled baseline, showing that most algorithms were able to recover the true structure when cluster separability was high, but failed when the noise dimensions or clustering distributions overlapped. The K-Means, agglomerative clustering, and GMM are comparatively stable against the noise introduced in datasets, while DBSCAN is severely affected and performs poorly in high-dimensional settings with large amounts of noise.

Using dimensionality reduction on the Spotify dataset is crucial. K-Means, agglomerative clustering, and GMM performed better on low-dimensional PCA representation, whereas DBSCAN was sensitive to parameter selection. The clusters formed both in K-Means and GMM are compact and interpretable, grouping the tracks into intelligible clusters, such as energetic pop, acoustic music, and live or speech-intensive tracks.

The MNIST dataset was the most challenging due to high dimensionality and overlapping digit classes. In this case, GMM with full covariance structure and moderate PCA reduction produced the best scores on external validation metrics compared to K-Means and agglomerative clustering. The BSCAN, however, was not very effective at generating meaningful clusters in the high-dimensional, sparse space.

Through all experiments, some general conclusions can be drawn. The application of dimensionality reduction is often necessary to reduce the curse of dimensionality and increase the reliability of distance-based measures. The choice of algorithms is closely dependent on dataset properties. K-Means is effective and efficient in the cases of spherical concentration and low- or medium-dimensional data. GMM has more flexibility and works better in overlapping or elliptical structures, but needs more computation. Agglomerative clustering is more interpretable but cannot be computed on large datasets. DBSCAN is applicable for spatial or density-based clustering in low-dimensional data, but unstable in high-dimensional and sparse data

Combined, these findings gave a global overview of where classical clustering algorithms can be used, the trade-offs involved, and how dataset properties should inform the choice of approach.

7.2 Limitations & Future Work

Although the study revealed informative insights, there were several limitations. To begin with, the implementation of DBSCAN was constrained by parameter sensitivity. Despite the use of grid searches, the selection of ε and `minPts` had a significant impact, especially in high-dimensional data where density estimates were rendered unreliable. Future work would involve adaptive parameter selection or other density-based methods such as DBSCAN++ and HDBSCAN. Secondly, due to computational limitations, only a subsample of the dataset was able to undergo agglomerative clustering. This restricted the direct comparability of results between algorithms. Alternative approaches, such as scalable hierarchical techniques or parallel implementations, would overcome this issue and enable the inclusion of complete datasets.

Furthermore, dimensionality reduction was performed via PCA alone. Although effective, PCA is a linear method with a tendency to miss out on non-linear constructs. More advanced techniques, such as t-SNE, UMAP, or autoencoders, can be considered to capture manifold structures, improving the quality of the cluster. Lastly, only classical clustering techniques were implemented. Recent developments in deep clustering and self-supervised representation learning have demonstrated good performance on high-dimensional data. Including such modern techniques in the analysis would contribute to a broader perspective and possibly eliminate some of the discovered limitations.

To conclude, though this paper has elucidated the advantages and drawbacks of four of the most popular clustering algorithms, further studies are necessary in the areas of adaptive density-based applications, scalable hierarchical clustering, non-linear manifold learning, and current neural clustering models. Such extensions would expand on the current results and provide more robust solutions to clustering in more complex, large-scale, and high-dimensional settings.

Bibliography

Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001). On the surprising behavior of distance metrics in high dimensional space. In *Proceedings of the International Conference on Database Theory (ICDT 2001)* (pp. 420–434). Springer. https://doi.org/10.1007/3-540-44503-X_27

Amelio, A., & Pizzuti, C. (2015). Is normalized mutual information a fair measure for comparing community detection methods? In *Proceedings of the 2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, 1584–1590. https://doi.org/10.1145/2808797.2809344

Arbelaitz, O., Gurrutxaga, I., Muguerza, J., Pérez, J. M., & Perona, I. (2013). An extensive comparative study of cluster validity indices. *Pattern Recognition*, **46**(1), 243–256. https://doi.org/10.1016/j.patcog.2012.07.021

Arthur, D., & Vassilvitskii, S. (2006). k-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)* (pp. 1027–1035). SIAM.

Bindra, K., & Mishra, A. (2017). A detailed study of clustering algorithms. In *Proceedings of the 2017 6th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)* (pp. 371–376). IEEE. https://doi.org/10.1109/ICRITO.2017.8342452

Bholowalia, P., & Kumar, A. (2014). EBK-Means: A Clustering Technique Based on Elbow Method and K-Means in WSN. *International Journal of Computer Applications*, **105**(9), 17–24.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Calinski, T., & Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics - Theory and Methods*, **3**(1), 1–27. https://doi.org/10.1080/03610927408827101

Coleman, G. B., & Andrews, H. C. (1979). Image segmentation by clustering. *Proceedings of the IEEE*, **67**(5), 773–785. https://doi.org/10.1109/PROC.1979.11327

Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **1**(2), 224–227. https://doi.org/10.1109/TPAMI.1979.4766909

Deng, H., & Han, J. (2018). Probabilistic models for clustering. In C. H. Chen, & R. L. R. St. Clair (Eds.), *Data Clustering* (pp. 61–86). Chapman and Hall/CRC. https://doi.org/10.1201/9781315189800-4

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD'96)* (pp. 226–231). AAAI Press.

Hamidani, Z. (2019). Spotify Tracks DB. Kaggle. Retrieved from https://www.kaggle.com/datasets/zaheenhamidani/ultimate-spotify-tracks-db (Accessed: 2025-06-25).

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, **2**(1), 193–218. https://doi.org/10.1007/BF01908075

Humaira, H., & Rasyidah, R. (2018). Determining the Appropriate Cluster Number Using Elbow Method for K-Means Algorithm. *WMA-2 2018, Padang, Indonesia*. DOI: 10.4108/eai.24-1-2018.2292388

Jain, A. K., Murty, M. N., & Flynn, P. J. (1999). Data clustering: A review. *ACM Computing Surveys*, **31**(3), 264–323. https://doi.org/10.1145/331499.331504

Jain, A. K. (2010). Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, **31**(8), 651–666. https://doi.org/10.1016/j.patrec.2009.09.011

Kaufman, L., & Rousseeuw, P. J. (2009). *Finding Groups in Data: An Introduction to Cluster Analysis*. John Wiley & Sons. https://doi.org/10.1002/9780470316801

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, **86**(11), 2278–2324. https://doi.org/10.1109/5.726791

Liu, Y., Wang, Y., Shenoy, A., Tsai, W.-H., & Cai, L. (2008). Clustering music recordings by their keys. In *Proceedings of the 9th International Society for Music Information Retrieval Conference (ISMIR)* (pp. 319–324).

Liu, Y., Li, Z., Xiong, H., Gao, X., & Wu, J. (2010). Understanding of internal clustering validation measures. In *Proceedings of the 2010 IEEE International Conference on Data Mining (ICDM)* (pp. 911–916). IEEE. https://doi.org/10.1109/ICDM.2010.35

MacQueen, J. B. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability*, (pp. 281–297).

McDaid, A. F., Greene, D., & Hurley, N. (2011). Normalized mutual information to evaluate overlapping community finding algorithms. *CoRR*, arXiv:1110.2515.

McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*. https://arxiv.org/abs/1802.03426

McLachlan, G. J., Lee, S. X., & Rathnayake, S. I. (2019). Finite mixture models. *Annual Review of Statistics and Its Application*, **6**(1), 355–378. https://doi.org/10.1146/annurev-statistics-031017-100325

Meilă, M. (2007). Comparing clusterings—an information based distance. *Journal of Multivariate Analysis*, **98**(5), 873–895.

Milligan, G. W., & Cooper, M. C. (1985). An examination of procedures for determining the number of clusters in a data set. *Psychometrika*, **50**(2), 159–179. https://doi.org/10.1007/BF02294245

Mirkin, B. (2011). Choosing the number of clusters. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, **1**(3), 252–260. https://doi.org/10.1002/widm.18

Murtagh, F., & Contreras, P. (2012). Algorithms for hierarchical clustering: An overview. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, **2**(1), 86–97. https://doi.org/10.1002/widm.53

Omran, M. G. H., Engelbrecht, A. P., & Salman, A. (2007). An overview of clustering methods. *Intelligent Data Analysis*, **11**(6), 583–605. https://doi.org/10.3233/IDA-2007-11602

Rand, W. M. (1971). Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical Association*, **66**(336), 846–850. https://doi.org/10.1080/01621459.1971.10482356

Rokach, L., & Maimon, O. (2005). Clustering methods. In O. Maimon & L. Rokach (Eds.), *Data Mining and Knowledge Discovery Handbook* (pp. 321–352). Springer. https://doi.org/10.1007/0-387-25465-X_15

Ros, F., Riad, R., & Guillaume, S. (2023). PDBI: A partitioning Davies-Bouldin index for clustering evaluation. *Neurocomputing*, **528**, 178–199. https://doi.org/10.1016/j.neucom.2022.11.051

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, **20**, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

Syakur, M. A., Khotimah, B. K., Rochman, E. M. S., & Satoto, B. D. (2018). Integration K-Means Clustering Method and Elbow Method for Identification of the Best Customer Profile Cluster. *IOP Conference Series: Materials Science and Engineering*, **336**(1), 012017. https://doi.org/10.1088/1757-899X/336/1/012017

Thorndike, R. L. (1953). Who belongs in the family? *Psychometrika*, **18**(4), 267–276.

Tibshirani, R., Walther, G., & Hastie, T. (2001). Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, **63**(2), 411–423. https://doi.org/10.1111/1467-9868.00293

van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, **9**, 2579–2605. http://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf

Vinh, N. X., Epps, J., & Bailey, J. (2010). Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research*, **11**, 2837–2854.

Wang, F., Franco-Peña, H.-H., Kelleher, J. D., Pugh, J., & Ross, R. (2017). An analysis of the application of simplified silhouette to the evaluation of k-means clustering validity. In *Proceedings of the International Conference on Machine Learning and Data Mining in Pattern Recognition (MLDM 2017)* (pp. 291–305). Springer. https://doi.org/10.1007/978-3-319-62416-7_23

Wattenberg, M., Viégas, F., & Johnson, I. (2016). How to use t-SNE effectively. *Distill*, **1**(10), e2. https://doi.org/10.23915/distill.00002

Wedel, M., & Kamakura, W. A. (2000). *Market segmentation: Conceptual and methodological foundations*. Springer Science & Business Media.

Xu, L., & Jordan, M. I. (1996). On convergence properties of the EM algorithm for Gaussian mixtures. *Neural Computation*, **8**(1), 129–151. https://doi.org/10.1162/neco.1996.8.1.129

Xu, D., & Tian, Y. (2015). A comprehensive survey of clustering algorithms. *Annals of Data Science*, **2**(2), 165–193. https://doi.org/10.1007/s40745-015-0040-1

Xu, R., & Wunsch, D. (2005). Survey of clustering algorithms. *IEEE Transactions on Neural Networks*, **16**(3), 645–678. https://doi.org/10.1109/TNN.2005.845141

Xu, R., & Wunsch, D. (2008). *Clustering*. John Wiley & Sons. https://doi.org/10.1002/9780470382776

Yeung, K. Y., & Ruzzo, W. L. (2001). Principal component analysis for clustering gene expression data. *Bioinformatics*, **17**(9), 763–774. https://doi.org/10.1093/bioinformatics/17.9.763